ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH – CO5085

THỰC HIỆN BÀI TOÁN PHÂN LOẠI ẢNH TRÊN TẬP DỮ LIỆU CIFAR-10 BẰNG CÁC MÔ HÌNH HỌC SÂU

Giảng viên phụ trách: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Lớp – Nhóm: CH01 – DL4DL
Sinh viên thực hiện: Lê Tiến Đạt – 2310653
Nguyễn Lê Thảo Ly – 2312010

Thành phố Hồ Chí Minh, Tháng 11/2026

MỤC LỤC

1 GIỚI THIỆU **7**
1.1 Tổng quan về bài toán phân loại ảnh . . . 7
1.2 Mục tiêu của các bài thực hành . . . 9
1.3 Cấu trúc báo cáo . . . 10

2 TẬP DỮ LIỆU VÀ THIẾT LẬP THỰC NGHIỆM **11**
2.1 Giới thiệu tập dữ liệu . . . 11
2.2 Phân tích và trực quan hóa dữ liệu . . . 12
2.3 Phân chia tập huấn luyện, kiểm định và kiểm thử . . . 14
2.4 Tiền xử lý dữ liệu . . . 14
2.5 Quy trình thực nghiệm chung . . . 15
2.6 Các chỉ số đánh giá . . . 15
2.6.1 Độ chính xác . . . 16
2.6.2 Năng lực và chi phí mô hình . . . 16
2.6.3 Độ ổn định giữa các seed . . . 16
2.6.4 Phân tích lỗi . . . 16

3 BÀI TẬP E1 – PHÂN LOẠI ẢNH VỚI SOFTMAX, MLP VÀ CNN **17**
3.1 Kiến trúc các mô hình . . . 17
3.1.1 Mô hình phân loại Softmax (Softmax classifier) . . . 17
3.1.2 Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron – MLP) . . . 18
3.1.3 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) . . . 18
3.2 Huấn luyện và đánh giá . . . 19
3.2.1 Quy trình huấn luyện . . . 20
3.2.2 Các thí nghiệm . . . 20
3.2.3 Tiêu chí đánh giá . . . 20
3.2.4 Môi trường thực nghiệm . . . 21
3.3 Kết quả thực nghiệm . . . 21
3.3.1 So sánh độ chính xác và quy mô mô hình . . . 21

3.3.2 Quá trình hội tụ . . . 21
3.3.3 Phân tích lỗi phân loại . . . 23
3.3.3.1 Softmax . . . 24
3.3.3.2 MLP . . . 25
3.3.3.3 MiniVGG . . . 27
3.4 Nhận xét và thảo luận . . . 28

4 BÀI TẬP E2 – MULTI-HEAD SELF-ATTENTION VÀ TOKENIZATION 29
4.1 Kiến trúc và cài đặt mô hình . . . 29
4.1.1 Multi-Head Self-Attention tự xây dựng . . . 29
4.1.2 Multi-Head Self-Attention sử dụng PyTorch . . . 30
4.1.3 Mô hình phân loại ảnh sử dụng MSA . . . 30
4.2 Các phương pháp Tokenization . . . 30
4.3 Huấn luyện và đánh giá . . . 31
4.3.1 Quy trình huấn luyện . . . 31
4.3.2 Các thí nghiệm . . . 32
4.3.3 Tiêu chí đánh giá . . . 32
4.3.4 Môi trường thực nghiệm . . . 32
4.4 Kết quả thực nghiệm . . . 33
4.4.1 MSA tự viết và MSA của PyTorch . . . 33
4.4.2 So sánh các phương pháp tokenization . . . 34
4.4.3 Vai trò của positional embedding . . . 36
4.4.4 Trực quan hóa bản đồ chú ý . . . 36
4.4.5 Phân tích lỗi . . . 38
4.5 Nhận xét và thảo luận . . . 39

5 BÀI TẬP E3 – PHÂN LOẠI ẢNH VỚI LSTM VÀ GRU 41
5.1 Các phương pháp Tokenization . . . 41
5.2 Kiến trúc các mô hình . . . 42
5.2.1 Mô hình LSTM . . . 42
5.2.2 Mô hình GRU . . . 43
5.3 Huấn luyện và đánh giá . . . 44
5.3.1 Quy trình huấn luyện . . . 44
5.3.2 Các thí nghiệm . . . 45
5.3.3 Tiêu chí đánh giá . . . 45
5.3.4 Môi trường thực nghiệm . . . 46
5.4 Kết quả thực nghiệm . . . 46
5.4.1 So sánh các cấu hình LSTM/GRU . . . 46

5.4.2 So sánh với baseline MLP/CNN 48
5.4.3 Phân tích lỗi 48
5.5 Nhận xét và thảo luận 50

6 KẾT LUẬN **52**
6.1 Kết quả chính 52
6.2 Hạn chế 52

TÀI LIỆU THAM KHẢO **54**

A TÀI NGUYÊN VÀ KHẢ NĂNG TÁI LẬP **55**
A.1 Tái lập kết quả E1 55
A.2 Tái lập kết quả E2 56
A.3 Tái lập kết quả E3 57
A.4 Khai báo sử dụng công cụ AI 58

DANH SÁCH BẢNG

2.1 Thông tin tổng quan về tập dữ liệu CIFAR-10 11
2.2 Phân chia tập dữ liệu trong thực nghiệm 14

3.1 Cấu hình huấn luyện của E1. 20
3.2 Độ chính xác kiểm thử, số tham số và chi phí huấn luyện của ba mô hình E1. Thời gian huấn luyện (s/ep.) là trung bình qua các seed của thời gian trung vị mỗi epoch; GPU là bộ nhớ đỉnh được ghi nhận. 21

4.1 Bốn cách tokenization được triển khai. 31
4.2 Cấu hình huấn luyện của E2. 32
4.3 Độ chính xác kiểm thử (%) và thời gian huấn luyện mỗi epoch (s/ep.) của hai bản MSA. 33
4.4 So sánh bốn tokenizer (MSA tự viết). Độ chính xác validation và kiểm thử (%); thời gian là trung vị giây/epoch huấn luyện; bộ nhớ là đỉnh GPU khi huấn luyện. 34
4.5 Chi phí tính toán một lượt thuận (MSA tự viết). MACs đếm các phép nhân ma trận (Linear, $QK^\top$, AV, tích chập) và khớp với công thức giải tích; ms/bước đo huấn luyện đầy đủ trên RTX 3050 Laptop. 34
4.6 Ablation positional embedding: độ chính xác kiểm thử (%). 36

5.1 Hai phương pháp tokenization được triển khai. 41
5.2 Cấu hình huấn luyện của E3. 45
5.3 Kết quả thực nghiệm của bốn cấu hình LSTM/GRU. 46
5.4 So sánh test accuracy và số tham số của các mô hình LSTM/GRU với baseline MLP và MiniVGG từ E1. Accuracy được báo cáo dưới dạng trung bình ± độ lệch chuẩn mẫu trên ba seed. 48

A.1 Liên kết GitHub Pages và notebook theo từng bài tập 55

DANH SÁCH HÌNH VẼ

2.1 Phân bố số lượng mẫu theo lớp trong tập huấn luyện CIFAR-10 12
2.2 Một số mẫu ảnh trong tập dữ liệu CIFAR-10 13

3.1 Kiến trúc MLP sử dụng trong E1. Ảnh đầu vào được làm phẳng thành vector 3072 chiều, sau đó đi qua hai khối ẩn tuyến tính kèm ReLU trước khi được ánh xạ sang 10 logits đầu ra. 18
3.2 Kiến trúc MiniVGG sử dụng trong E1. Sơ đồ tổng thể mô tả quá trình biến đổi đặc trưng từ ảnh đầu vào đến lớp phân loại; phần chi tiết trình bày cấu trúc dùng chung của hai khối tích chập. 19
3.3 Đường cong loss (trái) và accuracy (phải) trên tập huấn luyện và kiểm định của ba mô hình E1 qua 30 epoch. Các đường biểu diễn giá trị trung bình trên ba seed (0, 1, 2), với vùng tô thể hiện ± độ lệch chuẩn mẫu. 22
3.4 Phân tích lỗi của SoftmaxClassifier trên tập kiểm thử CIFAR-10 (seed 0). 24
3.5 Phân tích lỗi của MLP trên tập kiểm thử CIFAR-10 (seed 0). 25
3.6 Phân tích lỗi của MiniVGG trên tập kiểm thử CIFAR-10 (seed 0). 27

4.1 Sơ đồ mô hình phân loại ảnh dùng MSA. Chỉ khối MSA (tô xanh) được hiện thực hai cách; các phần còn lại giống hệt nhau. 30
4.2 Độ chính xác validation theo epoch của MSA tự viết (nét liền) và PyTorch (nét đứt), trung bình ba seed. 34
4.3 Loss huấn luyện (trái) và độ chính xác validation (phải) theo epoch của bốn tokenizer; đường là trung bình, vùng mờ là ± std ba seed. 35
4.4 Chú ý của `[CLS]` lên ảnh đầu vào: rollout và khối cuối cho từng tokenizer (seed 0). 37
4.5 Ma trận nhầm lẫn (CNN-stem, seed 0; % theo hàng). 38
4.6 Các ảnh dự đoán sai với độ tin cậy cao nhất. 39

5.1 Kiến trúc phân loại *many-to-one* sử dụng hidden state cuối cùng. 42
5.2 Cấu trúc bên trong một ô LSTM. Nguồn: Dive into Deep Learning [9]. 43
5.3 Cấu trúc GRU cell. Nguồn: Dive into Deep Learning [8]. 44

5.4 Diễn biến training loss (trái) và validation accuracy (phải) của bốn cấu hình LSTM/GRU qua 30 epoch huấn luyện trên CIFAR-10. Đường biểu diễn giá trị trung bình trên ba seed;vùng tô mờ xung quanh thể hiện độ lệch chuẩn. Các cấu hình được phân biệt theo recurrent cell (LSTM, GRU) và phương pháp biểu diễn chuỗi (Row, Patch4). 47
5.5 Ma trận nhầm lẫn của GRU-Patch4 (seed 2) trên tập test CIFAR-10, chuẩn hóa theo từng lớp thực tế. Hàng biểu diễn nhãn thực, cột biểu diễn nhãn dự đoán; các giá trị trong ma trận là tỷ lệ phần trăm được làm tròn. 49
5.6 Mười hai ảnh dự đoán sai có độ tin cậy cao của GRU-Patch4 (seed 2) trên tập test CIFAR-10. Mỗi ảnh hiển thị nhãn thực (True), nhãn dự đoán (Pred) và xác suất dự đoán của mô hình. 50

Chương 1

GIỚI THIỆU

1.1 Tổng quan về bài toán phân loại ảnh

Phân loại ảnh (*Image Classification*) là một trong những bài toán cơ bản của thị giác máy tính (*Computer Vision*), với mục tiêu xác định nhãn lớp tương ứng cho một ảnh đầu vào. Bài toán này thuộc nhóm học máy có giám sát (*Supervised Learning*), trong đó mô hình được huấn luyện dựa trên tập dữ liệu gồm các mẫu đầu vào và nhãn tương ứng.

Một tập dữ liệu có giám sát được biểu diễn dưới dạng:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}, \tag{1.1}$$

trong đó N là số lượng mẫu dữ liệu, x_i là đầu vào và y_i là nhãn thực tế (*ground-truth label*) của mẫu thứ i.

Mục tiêu của học máy là học một ánh xạ từ không gian dữ liệu đầu vào đến không gian đầu ra, thay vì xây dựng các quy tắc dự đoán một cách tường minh. Ánh xạ này được biểu diễn tổng quát như sau:

$$f : x \mapsto y. \tag{1.2}$$

Trong đó, hàm f được xấp xỉ thông qua một mô hình có tham số f_θ, với θ là tập hợp các tham số được học từ dữ liệu. Mục tiêu của quá trình huấn luyện là tìm được bộ tham số sao cho đầu ra dự đoán của mô hình phù hợp với nhãn thực tế:

$$\hat{y} = f_\theta(x) \approx y. \tag{1.3}$$

Tùy theo đặc điểm của không gian đầu ra, các bài toán học có giám sát có thể được chia thành hai nhóm phổ biến là phân loại (*Classification*) và hồi quy (*Regression*). Đối với bài toán phân

loại, nhãn đầu ra thuộc một tập hữu hạn gồm K lớp:

$$y \in \{1, 2, \ldots, K\}. \tag{1.4}$$

Các bài toán phân loại thường được chia thành phân loại nhị phân (*Binary Classification*), phân loại đa lớp với một nhãn cho mỗi mẫu (*Single-label Multi-class Classification*) và phân loại đa nhãn (*Multi-label Classification*). Trong phạm vi các bài thực hành, bài toán được xem xét là phân loại đa lớp với một nhãn cho mỗi ảnh.

Đối với dữ liệu ảnh, mỗi mẫu đầu vào x_i được biểu diễn dưới dạng một tensor:

$$x_i \in \mathbb{R}^{C \times H \times W}, \tag{1.5}$$

trong đó C, H và W lần lượt biểu thị số kênh màu, chiều cao và chiều rộng của ảnh.

Khác với bài toán hồi quy, mô hình phân loại đa lớp có thể đưa ra một phân phối xác suất trên tập các lớp. Xác suất có điều kiện để một mẫu đầu vào x thuộc lớp k được biểu diễn bởi:

$$P(y = k \mid x), \quad k = 1, 2, \ldots, K. \tag{1.6}$$

Trong cách tiếp cận sử dụng mạng nơ-ron, một mô hình tham số hóa g_θ tạo ra vector giá trị dự đoán chưa chuẩn hóa (*logits*):

$$\mathbf{z} = g_\theta(x) \in \mathbb{R}^K. \tag{1.7}$$

Đối với bài toán phân loại đa lớp với một nhãn, hàm *Softmax* được sử dụng để chuyển đổi logits thành phân phối xác suất dự đoán:

$$\hat{y}_k = \hat{p}_k = P_\theta(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \quad k = 1, 2, \ldots, K. \tag{1.8}$$

Các xác suất dự đoán thỏa mãn $\hat{p}_k \geq 0$ và $\sum_{k=1}^{K} \hat{p}_k = 1$.

Quá trình huấn luyện mô hình là một bài toán tối ưu hóa, trong đó các tham số θ được điều chỉnh nhằm cực tiểu hóa hàm mất mát (*Loss Function*) trên tập dữ liệu huấn luyện. Với bài toán phân loại đa lớp một nhãn, một lựa chọn phổ biến là hàm mất mát *Cross-Entropy*:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{i,k} \log \hat{y}_{i,k} \tag{1.9}$$

trong đó $\hat{p}_{i,y_i}$ là xác suất mà mô hình dự đoán cho lớp thực tế của mẫu thứ i. Bộ tham số tối ưu được xác định theo:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta). \tag{1.10}$$

Đối với bài toán phân loại, độ chính xác (*Accuracy*) là một trong những chỉ số đánh giá phổ biến, bên cạnh *Precision*, *Recall* và *F1-score*.

Trên cơ sở đó, các bài thực hành E1–E3 tập trung triển khai, huấn luyện và đánh giá những kiến trúc học sâu khác nhau trên cùng một bài toán phân loại ảnh, nhằm khảo sát ảnh hưởng của cách biểu diễn dữ liệu và đặc điểm kiến trúc đến hiệu quả phân loại và khả năng tổng quát hóa của mô hình.

1.2 Mục tiêu của các bài thực hành

Các bài thực hành E1–E3 được thực hiện nhằm củng cố kiến thức về các kiến trúc học sâu thông qua quá trình xây dựng, huấn luyện và đánh giá mô hình trên bài toán phân loại ảnh có giám sát.

Các mục tiêu cụ thể bao gồm:

- **E1 – Phân loại ảnh bằng Softmax, MLP và CNN:** Xây dựng và huấn luyện ba mô hình phân loại ảnh gồm bộ phân loại Softmax, mạng truyền thẳng nhiều lớp (MLP) và mạng nơ-ron tích chập (CNN). Thông qua việc so sánh độ chính xác, số lượng tham số, quá trình hội tụ và các trường hợp dự đoán sai, bài thực hành nhằm khảo sát sự khác biệt giữa mô hình tuyến tính, mạng truyền thẳng phi tuyến và mạng khai thác cấu trúc không gian của ảnh.

- **E2 – Multi-Head Self-Attention và Tokenization:** Tìm hiểu và hiện thực cơ chế *Multi-Head Self-Attention* (MSA) từ các phép toán cơ bản, đồng thời xây dựng phiên bản sử dụng module có sẵn trong PyTorch để đối chiếu. Tích hợp cơ chế MSA vào mô hình phân loại ảnh, khảo sát ít nhất hai phương pháp chuyển đổi ảnh thành chuỗi token và đánh giá ảnh hưởng của cách biểu diễn token đến độ chính xác và thời gian thực thi.

- **E3 – Phân loại ảnh bằng LSTM và GRU:** Xây dựng mô hình phân loại ảnh dựa trên hai kiến trúc mạng hồi quy *Long Short-Term Memory* (LSTM) và *Gated Recurrent Unit* (GRU). Ảnh đầu vào được chuyển đổi thành chuỗi các thành phần nhằm khảo sát khả năng mô hình hóa quan hệ phụ thuộc trong dữ liệu. Kết quả của hai kiến trúc được đánh giá và so sánh với các mô hình cơ sở từ E1.

Ngoài các mục tiêu riêng của từng bài, quá trình thực hành còn hướng đến việc xây dựng quy trình thực nghiệm nhất quán, bao gồm chuẩn bị dữ liệu, triển khai vòng lặp huấn luyện bằng PyTorch, đánh giá trên các tập dữ liệu độc lập và phân tích kết quả. Các thiết lập huấn luyện và kết quả thực nghiệm được ghi nhận nhằm hỗ trợ việc so sánh và tái lập thí nghiệm.

1.3 Cấu trúc báo cáo

Báo cáo được tổ chức theo trình tự từ phát biểu bài toán, chuẩn bị dữ liệu và thiết lập thực nghiệm đến triển khai mô hình, đánh giá và phân tích kết quả. Nội dung được phân chia thành các phần chính như sau:

- **Giới thiệu:** Trình bày tổng quan về bài toán phân loại ảnh có giám sát, mục tiêu của các bài thực hành và cấu trúc báo cáo.
- **Tập dữ liệu và thiết lập thực nghiệm:** Giới thiệu tập dữ liệu được sử dụng, phương pháp tiền xử lý, cách phân chia tập huấn luyện, xác thực và kiểm tra. Đồng thời, trình bày các thiết lập chung và chỉ số đánh giá phục vụ quá trình thực nghiệm.
- **Bài thực hành E1 – Softmax, MLP và CNN:** Trình bày kiến trúc các mô hình phân loại, phương pháp huấn luyện và kết quả thực nghiệm. Tiến hành so sánh và phân tích sự khác biệt về hiệu quả phân loại và độ phức tạp mô hình.
- **Bài thực hành E2 – Multi-Head Self-Attention và Tokenization:** Trình bày cách hiện thực cơ chế tự chú ý đa đầu, các phương pháp biểu diễn ảnh thành token và kết quả so sánh giữa những phương pháp được triển khai.
- **Bài thực hành E3 – LSTM và GRU:** Trình bày phương pháp biểu diễn ảnh dưới dạng chuỗi, kiến trúc mạng hồi quy và kết quả thực nghiệm so sánh với các mô hình cơ sở.
- **Kết luận:** Tổng hợp các kết quả chính, thảo luận những nhận xét rút ra từ các bài thực hành và chỉ ra những hạn chế của quá trình thực nghiệm.

Chương 2

TẬP DỮ LIỆU VÀ THIẾT LẬP THỰC NGHIỆM

2.1 Giới thiệu tập dữ liệu

Trong bài tập này, nhóm sử dụng tập dữ liệu CIFAR-10 [3] cho bài toán phân loại ảnh đa lớp. CIFAR-10 gồm 60.000 ảnh màu có kích thước 32×32 pixel, được chia thành 10 lớp khác nhau. Mỗi ảnh có ba kênh màu RGB.

Tập dữ liệu gốc gồm 50,000 ảnh dùng cho huấn luyện và 10,000 ảnh dùng cho kiểm thử. Mỗi lớp có số lượng mẫu bằng nhau, với 6,000 ảnh mỗi lớp (tương đương 5.000 ảnh huấn luyện và 1.000 ảnh kiểm thử).

Mười lớp của CIFAR-10 gồm: *airplane*, *automobile*, *bird*, *cat*, *deer*, *dog*, *frog*, *horse*, *ship* và *truck*.

Thuộc tính	**Giá trị**
Tổng số ảnh	60,000
Số ảnh huấn luyện gốc	50,000
Số ảnh kiểm thử	10,000
Kích thước ảnh	32×32
Số kênh màu	3 (RGB)
Số lớp	10

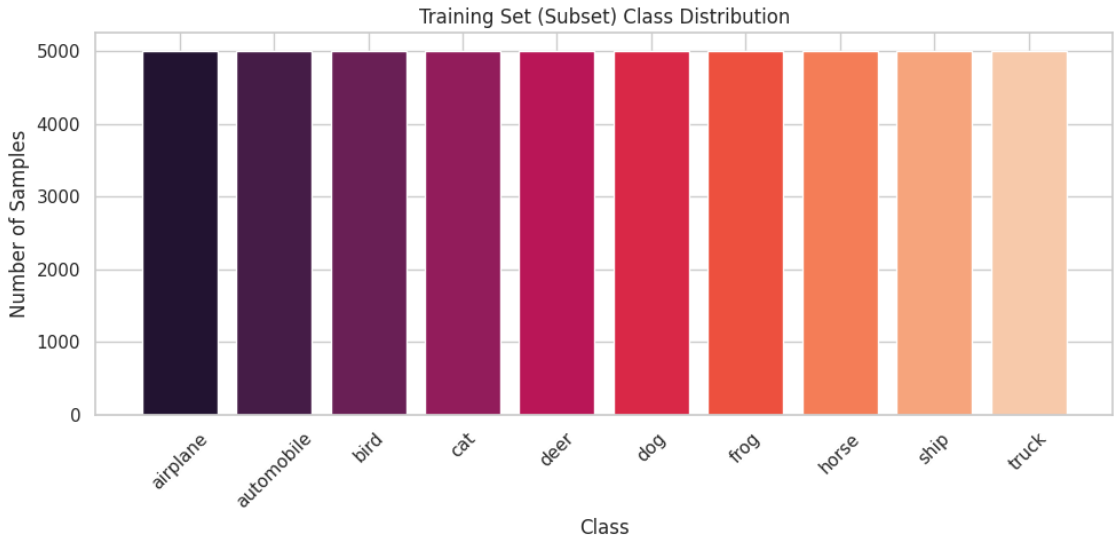
Bảng 2.1: Thông tin tổng quan về tập dữ liệu CIFAR-10

CIFAR-10 được lựa chọn vì quy mô dữ liệu phù hợp với phạm vi bài tập và khả năng tính toán, đồng thời vẫn có độ phức tạp đủ lớn để làm rõ sự khác biệt về hiệu suất giữa các phương pháp phân loại.

2.2 Phân tích và trực quan hóa dữ liệu

Trước khi huấn luyện mô hình, dữ liệu CIFAR-10 được khảo sát thông qua phân bố số lượng mẫu của từng lớp và in ra một số mẫu đại diện cho mỗi lớp.

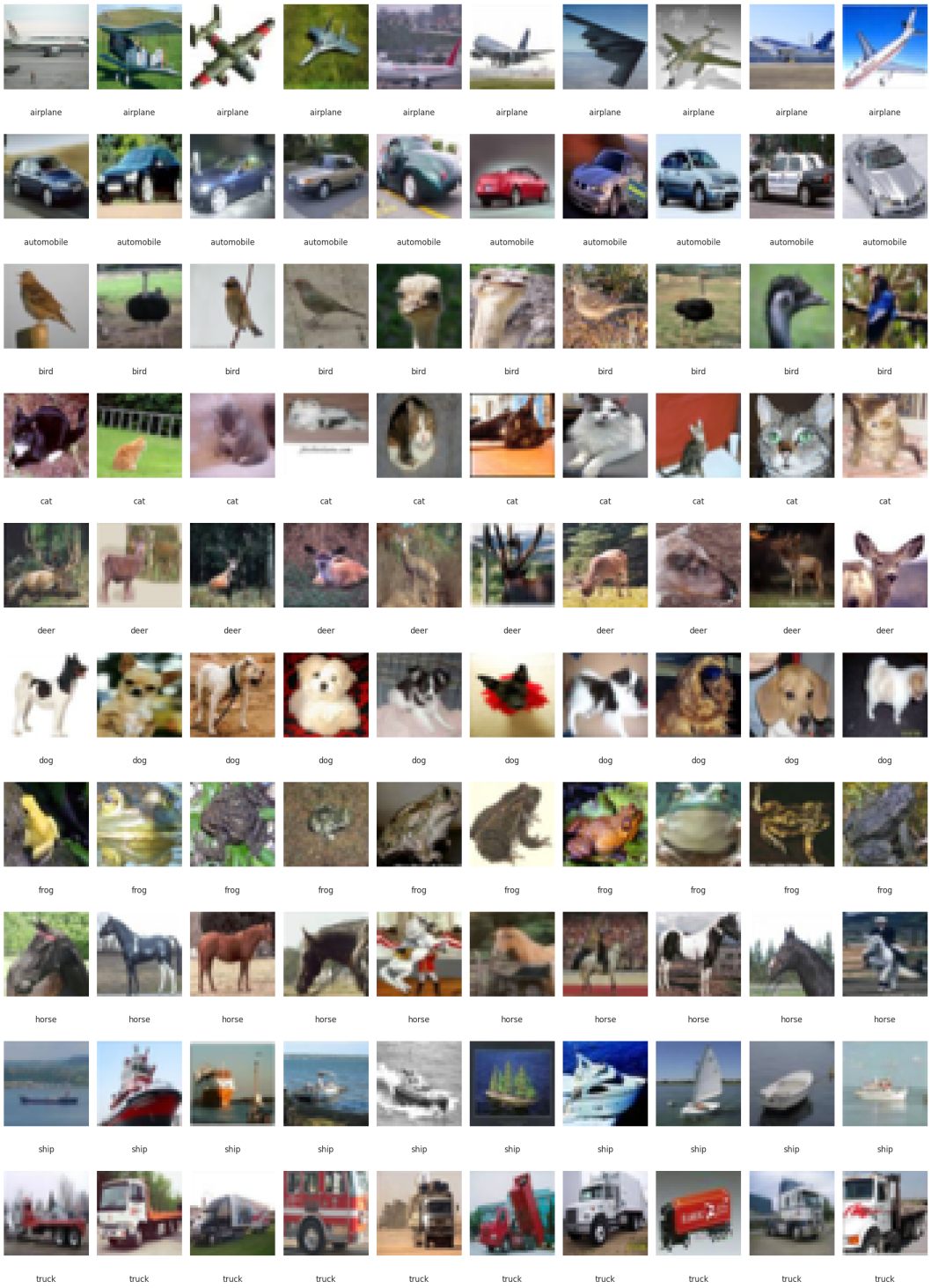
Mục đích của bước này là quan sát đặc điểm trực quan của dữ liệu và kiểm tra mức độ cân bằng giữa các lớp. Hình 2.1 trình bày số lượng mẫu theo từng lớp trong tập huấn luyện gốc.



Hình 2.1: Phân bố số lượng mẫu theo lớp trong tập huấn luyện CIFAR-10

Có thể thấy 10 lớp có số lượng mẫu bằng nhau, với 5.000 ảnh cho mỗi lớp trong tập huấn luyện gốc. CIFAR-10 là tập dữ liệu cân bằng và không cần áp dụng các kỹ thuật xử lý mất cân bằng lớp.

Hình 2.2 minh họa một số ảnh thuộc các lớp khác nhau trong CIFAR-10.



Hình 2.2: Một số mẫu ảnh trong tập dữ liệu CIFAR-10

Các ảnh trong CIFAR-10 có độ phân giải thấp, nhưng vẫn thể hiện sự đa dạng về màu sắc, hình dạng đối tượng và bối cảnh nền. Một số lớp có đặc điểm thị giác tương đối giống nhau, chẳng hạn *cat* và *dog*, hoặc *automobile* và *truck*, làm tăng độ khó của bài toán phân loại.

2.3 Phân chia tập huấn luyện, kiểm định và kiểm thử

Tập huấn luyện gốc của CIFAR-10 gồm 50 000 ảnh được chia *một lần* thành 45 000 ảnh huấn luyện và 5 000 ảnh kiểm định theo kiểu phân tầng: mỗi lớp giữ lại đúng 500 ảnh cho tập kiểm định, nên cả hai tập đều cân bằng lớp. Phép chia dùng seed cố định 42, các chỉ số ảnh được lưu trong tệp `cifar10_split.json` (thư mục `common`) cùng mã băm SHA-256, và mọi bài tập (E1, E2, E3) đọc *cùng một* tệp này; mỗi kết quả huấn luyện ghi lại mã băm để kiểm tra các lần chạy dùng cùng một phép chia. Dữ liệu được tải qua thư viện `datasets` của Hugging Face (`uoft-cs/cifar10`) và chỉ được chấp nhận khi mã băm SHA-256 của toàn bộ ảnh và nhãn trùng với bản TorchVision dùng để tạo phép chia; bản Hugging Face chứa cùng 50 000 ảnh huấn luyện nhưng theo thứ tự khác, nên được sắp xếp lại về thứ tự TorchVision trước khi áp dụng các chỉ số đã lưu. Tập kiểm thử gốc gồm 10 000 ảnh được giữ nguyên, không dùng để chọn mô hình hay siêu tham số, và chỉ được đánh giá sau khi huấn luyện xong.

Bảng 2.2 trình bày cách phân chia dữ liệu trong thực nghiệm.

Tập dữ liệu	Số mẫu	Mục đích
Training	45 000	Huấn luyện mô hình
Validation	5 000	Theo dõi quá trình huấn luyện
Test	10 000	Đánh giá kết quả cuối cùng

Bảng 2.2: Phân chia tập dữ liệu trong thực nghiệm

2.4 Tiền xử lý dữ liệu

Trước khi đưa dữ liệu vào mô hình, các ảnh CIFAR-10 được chuyển đổi và chuẩn hóa nhằm bảo đảm đầu vào có định dạng phù hợp cho quá trình huấn luyện.

Đầu tiên, mỗi ảnh được chuyển sang tensor PyTorch và giá trị pixel được ánh xạ từ khoảng $[0, 255]$ về khoảng $[0, 1]$.

Sau khi chia tập huấn luyện gốc thành 45 000 ảnh huấn luyện và 5 000 ảnh kiểm định, giá trị trung bình và độ lệch chuẩn được tính riêng cho từng kênh màu *chỉ trên 45 000 ảnh huấn luyện*, để thông tin của tập kiểm định và kiểm thử không rò rỉ vào bước chuẩn hóa. Gọi μ_c và σ_c lần lượt là giá trị trung bình và độ lệch chuẩn của kênh màu c. Mỗi giá trị pixel x_c được chuẩn hóa theo công thức:

$$x'_c = \frac{x_c - \mu_c}{\sigma_c}, \tag{2.1}$$

trong đó $c \in \{R, G, B\}$. Các giá trị thu được là

$$\mu \approx (0{,}4918;\ 0{,}4825;\ 0{,}4468), \qquad \sigma \approx (0{,}2471;\ 0{,}2435;\ 0{,}2617).$$

Cùng một phép chuẩn hóa được áp dụng cho tập huấn luyện, tập kiểm định và tập kiểm thử.

Tăng cường dữ liệu (data augmentation). Chỉ trên tập huấn luyện, mỗi ảnh được cắt ngẫu nhiên 32×32 sau khi đệm 4 pixel (random crop) và lật ngang với xác suất 0,5; phép biến đổi được thực hiện sau chuẩn hóa. Tập kiểm định và kiểm thử không bị biến đổi. Các bài tập E1–E3 dùng cùng cấu hình augmentation để so sánh được với nhau.

2.5 Quy trình thực nghiệm chung

Để các bài tập có thể so sánh với nhau, mọi thực nghiệm tuân theo cùng một quy trình:

- **Dữ liệu:** cùng phép chia 45 000/5 000/10 000 và cùng thống kê chuẩn hóa (mục 2.1).
- **Seed:** mỗi cấu hình được huấn luyện với ba seed $0, 1, 2$ (seed điều khiển khởi tạo, thứ tự dữ liệu và augmentation); kết quả là trung bình $\pm$ độ lệch chuẩn mẫu.
- **Số epoch:** 30 epoch cho mọi mô hình của E1–E3.
- **Kết quả chính:** độ chính xác kiểm thử với trọng số của epoch *cuối*; không chọn checkpoint hay siêu tham số theo tập kiểm thử. (E1 ghi thêm kết quả của checkpoint có loss kiểm định thấp nhất.)
- **Tái lập:** mỗi lần chạy lưu một tệp JSON gồm cấu hình, mã băm phép chia, lịch sử huấn luyện, kết quả cuối, thời gian và thông tin môi trường; các bảng và hình trong báo cáo được sinh từ các tệp này bằng mã nguồn trong kho lưu trữ; mỗi bài tập có một notebook Colab nạp lại kết quả đã lưu, sinh lại bảng, hình và kiểm tra chúng khớp với báo cáo, hoặc huấn luyện lại từ đầu (Phụ lục A).

Các siêu tham số riêng của từng bài (bộ tối ưu, tốc độ học, kích thước batch) được nêu trong chương tương ứng.

2.6 Các chỉ số đánh giá

Bốn khía cạnh chính được dùng để so sánh các mô hình: độ chính xác phân loại, năng lực và chi phí mô hình, độ ổn định giữa các seed và các trường hợp dự đoán sai.

2.6.1 Độ chính xác

Độ chính xác là chỉ số chính:

$$\text{Accuracy} = \frac{\text{Số mẫu được dự đoán đúng}}{\text{Tổng số mẫu}} \times 100\%. \tag{2.2}$$

Do CIFAR-10 có phân bố lớp cân bằng, accuracy phù hợp để so sánh hiệu năng tổng thể. Độ chính xác kiểm thử được tính trên 10 000 ảnh; độ chính xác kiểm định chỉ dùng để theo dõi.

2.6.2 Năng lực và chi phí mô hình

Năng lực của mô hình được xem xét qua số tham số huấn luyện được $P = \sum_{i=1}^{L} |\theta_i|$, kết hợp với đặc điểm kiến trúc. Chi phí tính toán được đánh giá bằng thời gian huấn luyện trung vị mỗi epoch, bộ nhớ GPU đỉnh khi huấn luyện và thời gian suy luận (cùng số phép nhân-cộng cho một ảnh ở E2). Các số đo thời gian và bộ nhớ chỉ so sánh được giữa các mô hình chạy trên cùng một máy.

2.6.3 Độ ổn định giữa các seed

Mỗi cấu hình được chạy với ba seed; báo cáo trung bình $\pm$ độ lệch chuẩn mẫu ($n-1$ ở mẫu số). Với $n = 3$, độ lệch chuẩn chỉ là ước lượng thô, vì vậy chỉ các chênh lệch lớn hơn rõ rệt so với độ lệch chuẩn mới được xem là khác biệt thực sự.

2.6.4 Phân tích lỗi

Ma trận nhầm lẫn (chuẩn hóa theo hàng) được dùng để xác định các cặp lớp thường bị nhầm, và một số ảnh bị phân loại sai với độ tin cậy cao được trực quan hóa cùng nhãn thật và nhãn dự đoán để làm rõ kiểu lỗi đặc trưng của từng mô hình.

Chương 3

BÀI TẬP E1 – PHÂN LOẠI ẢNH VỚI SOFTMAX, MLP VÀ CNN

3.1 Kiến trúc các mô hình

Bài tập E1 khảo sát ba cách tiếp cận có mức độ phức tạp kiến trúc khác nhau cho bài toán phân loại ảnh CIFAR-10: bộ phân loại Softmax trên vector ảnh làm phẳng, mạng truyền thẳng nhiều lớp MLP trên cùng biểu diễn đầu vào, và mạng nơ-ron tích chập CNN. Mục tiêu chính là so sánh khả năng phân loại, năng lực biểu diễn và đặc điểm lỗi của ba kiến trúc khi được huấn luyện theo cùng một quy trình thực nghiệm.

3.1.1 Mô hình phân loại Softmax (Softmax classifier)

Softmax classifier được sử dụng làm mô hình cơ sở đơn giản nhất. Ảnh đầu vào có kích thước $32 \times 32 \times 3$ được làm phẳng thành vector có 3072 phần tử. Vector này được ánh xạ trực tiếp sang 10 logits bằng một lớp tuyến tính:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \tag{3.1}$$

trong đó $\mathbf{x} \in \mathbb{R}^{3072}$ là ảnh sau khi làm phẳng, $W \in \mathbb{R}^{10\times3072}$ và $\mathbf{b} \in \mathbb{R}^{10}$ là các tham số có thể huấn luyện.

Mô hình không chứa lớp ẩn hoặc phép biến đổi phi tuyến. Do đó, đây là một bộ phân loại tuyến tính trên không gian pixel đầu vào. Tổng số tham số có thể huấn luyện của mô hình là 30 730.

3.1.2 Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron – MLP)

MLP mở rộng mô hình tuyến tính bằng hai lớp ẩn sử dụng hàm kích hoạt ReLU. Ảnh đầu vào tiếp tục được biểu diễn dưới dạng vector 3072 chiều, sau đó lần lượt đi qua các phép biến đổi:

$$3072 \to 64 \to 32 \to 10. \tag{3.2}$$

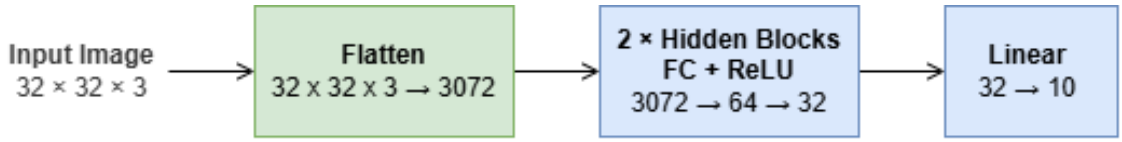
Hai lớp ẩn được xác định bởi:

$$\mathbf{h}_1 = \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1), \tag{3.3}$$
$$\mathbf{h}_2 = \mathrm{ReLU}(W_2\mathbf{h}_1 + \mathbf{b}_2). \tag{3.4}$$

Đầu ra cuối cùng được tính bằng một lớp tuyến tính:

$$\mathbf{z} = W_3\mathbf{h}_2 + \mathbf{b}_3. \tag{3.5}$$

Mô hình có **199.082 tham số có thể huấn luyện**. So với Softmax classifier, các lớp ẩn cho phép MLP học những quan hệ phi tuyến giữa các đặc trưng đầu vào. Tuy nhiên, kiến trúc vẫn sử dụng biểu diễn ảnh làm phẳng và không trực tiếp mã hóa tính cục bộ của các quan hệ không gian.



Hình 3.1: Kiến trúc MLP sử dụng trong E1. Ảnh đầu vào được làm phẳng thành vector 3072 chiều, sau đó đi qua hai khối ẩn tuyến tính kèm ReLU trước khi được ánh xạ sang 10 logits đầu ra.

3.1.3 Mạng nơ-ron tích chập (Convolutional Neural Network – CNN)

Mô hình CNN được xây dựng tham khảo kiến trúc MiniVGGNet do Rosebrock giới thiệu [6], với các điều chỉnh nhằm phù hợp quy mô thực nghiệm. Kiến trúc gồm hai khối tích chập liên tiếp và một lớp tuyến tính đầu ra. Mỗi khối sử dụng hai phép tích chập với kernel 3×3, kết hợp hàm kích hoạt ReLU, Batch Normalization và Max Pooling. So với kiến trúc tham khảo, mô hình được giảm số kênh đặc trưng xuống lần lượt 16 và 32, đồng thời sử dụng trực tiếp lớp tuyến tính sau bước làm phẳng đặc trưng.

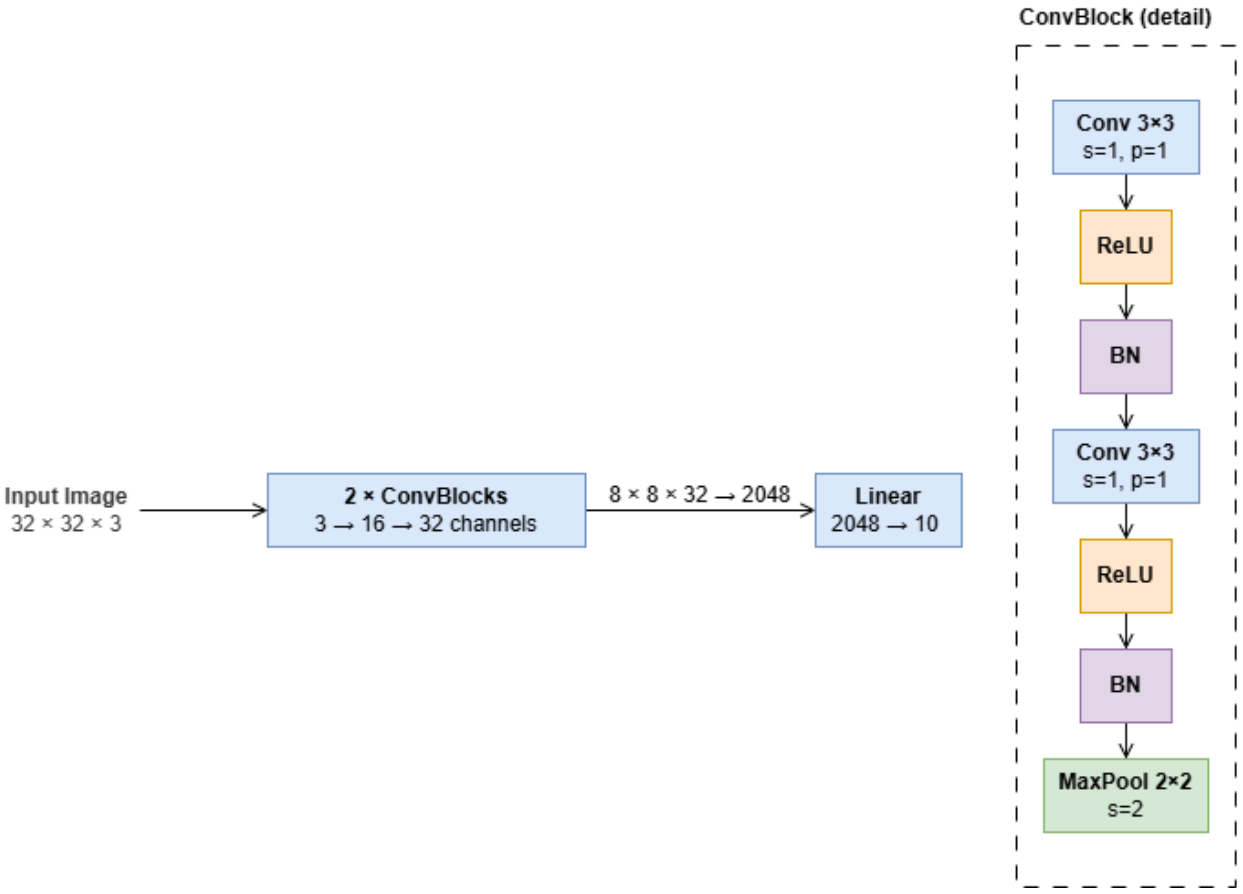
Hai khối lần lượt tạo ra 16 và 32 kênh đặc trưng. Quá trình biến đổi kích thước tensor, theo thứ tự $H \times W \times C$, được biểu diễn như sau:

$$32 \times 32 \times 3 \longrightarrow 16 \times 16 \times 16 \longrightarrow 8 \times 8 \times 32. \tag{3.6}$$

Đầu ra của khối tích chập thứ hai được làm phẳng thành vector 2048 chiều trước khi đưa qua lớp tuyến tính để tạo 10 logits:

$$\mathbf{z} = \text{Linear}(\text{Flatten}(\mathbf{F})), \qquad \mathbf{F} \in \mathbb{R}^{8\times8\times32}. \tag{3.7}$$

MiniVGG có tổng cộng **37.338 tham số có thể huấn luyện**. Việc sử dụng các bộ lọc cục bộ và chia sẻ trọng số cho phép mô hình khai thác thông tin không gian mà không cần kết nối đầy đủ giữa mọi phần tử của ảnh như trong MLP.



Hình 3.2: Kiến trúc MiniVGG sử dụng trong E1. Sơ đồ tổng thể mô tả quá trình biến đổi đặc trưng từ ảnh đầu vào đến lớp phân loại; phần chi tiết trình bày cấu trúc dùng chung của hai khối tích chập.

3.2 Huấn luyện và đánh giá

3.2.1 Quy trình huấn luyện

Ba mô hình E1 sử dụng cùng tập dữ liệu và quy trình tiền xử lý (phân chia dữ liệu, chuẩn hóa và augmentation theo mục 2.1–2.5). Cấu hình huấn luyện được tóm tắt ở Bảng 3.1; các siêu tham số được giữ cố định giữa ba mô hình nhằm tập trung đánh giá ảnh hưởng của kiến trúc, không tinh chỉnh riêng từng mô hình trên tập kiểm thử.

Thành phần	Thiết lập
Kiến trúc	Softmax, MLP, MiniVGG
Tối ưu	Adam, learning rate 10^{-3}
Lịch lr	Learning rate cố định
Hàm mất mát	Cross-entropy
Huấn luyện	30 epoch, batch size 64
Augmentation	Random crop (đệm 4) và lật ngang, chỉ trên tập huấn luyện
Seed	0, 1, 2 (ảnh hưởng khởi tạo, thứ tự dữ liệu và augmentation)

Bảng 3.1: Cấu hình huấn luyện của E1.

3.2.2 Các thí nghiệm

Tổng cộng có 9 lần huấn luyện: 3 kiến trúc (Softmax, MLP, MiniVGG) $\times$ 3 seed (0, 1, 2). Các mô hình được huấn luyện độc lập với cùng cấu hình và quy trình xử lý dữ liệu, nhằm so sánh ảnh hưởng của kiến trúc đến độ chính xác phân loại, số tham số, chi phí tính toán và đặc điểm dự đoán sai.

3.2.3 Tiêu chí đánh giá

Chỉ số chính là độ chính xác trên 10 000 ảnh kiểm thử chính thức, tính bằng trọng số của epoch cuối cùng, báo cáo dưới dạng trung bình $\pm$ độ lệch chuẩn mẫu trên 3 seed. Độ chính xác validation được sử dụng để theo dõi quá trình huấn luyện; checkpoint có validation loss thấp nhất được lưu để phân tích bổ sung, không thay thế kết quả chính.

Ngoài ra, báo cáo số tham số có thể huấn luyện, thời gian huấn luyện trung vị mỗi epoch, bộ nhớ GPU đỉnh và thời gian suy luận trên mỗi ảnh. Quá trình hội tụ được đánh giá thông qua đường cong loss và accuracy; các trường hợp dự đoán sai được phân tích bằng confusion matrix và những mẫu có độ tin cậy dự đoán cao. Độ biến thiên giữa các lần chạy được xem xét thông qua độ lệch chuẩn mẫu trên ba seed, nhằm đánh giá tính ổn định của kết quả đối với yếu tố ngẫu nhiên trong quá trình huấn luyện.

3.2.4 Môi trường thực nghiệm

E1 được thực hiện trên GPU NVIDIA GeForce RTX 3050 Laptop 6 GB, sử dụng Python 3.12.15, PyTorch 2.14.1 (CUDA 12.6). Các tệp kết quả ghi nhận cùng một môi trường phần mềm và phần cứng. Thời gian huấn luyện và mức sử dụng bộ nhớ chỉ có giá trị so sánh *tương đối* giữa các mô hình trong môi trường thực nghiệm này.

3.3 Kết quả thực nghiệm

Mọi kết quả dưới đây được báo cáo dưới dạng trung bình ± độ lệch chuẩn mẫu trên ba seed (0, 1, 2), tính bằng % trên tập kiểm thử chính thức tại epoch cuối, trừ khi có ghi chú khác. Số liệu của từng lần chạy được lưu tại `E1/results`; bảng tổng hợp được sinh bởi `python -m e1lib.summary`.

3.3.1 So sánh độ chính xác và quy mô mô hình

Bảng 3.2 tổng hợp độ chính xác kiểm thử, số tham số và chi phí huấn luyện của ba kiến trúc.

Mô hình	Tham số	Test accuracy (%)	s/ep.	GPU (MB)
Softmax	30 730	32,61 ± 0,98	7,4	17,7
MLP	199 082	51,17 ± 0,23	8,5	21,6
MiniVGG	37 338	79,62 ± 0,26	9,5	49,0

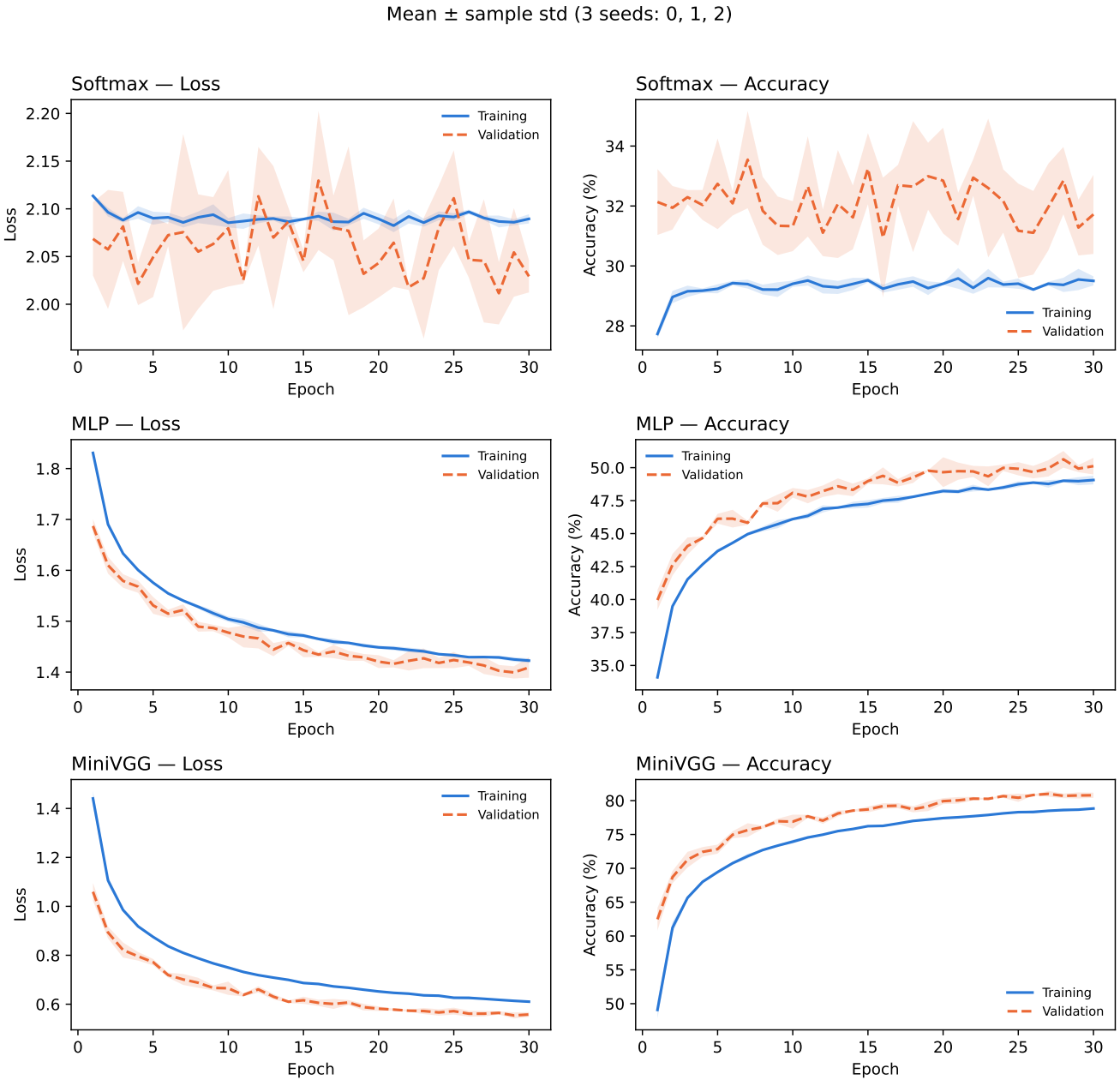
Bảng 3.2: Độ chính xác kiểm thử, số tham số và chi phí huấn luyện của ba mô hình E1. Thời gian huấn luyện (s/ep.) là trung bình qua các seed của thời gian trung vị mỗi epoch; GPU là bộ nhớ đỉnh được ghi nhận.

MLP đạt độ chính xác cao hơn Softmax xấp xỉ 18,6 điểm phần trăm, cho thấy lợi ích của các phép biến đổi phi tuyến trên cùng biểu diễn đầu vào. MiniVGG tiếp tục cải thiện 28,45 điểm phần trăm so với MLP, dù chỉ sử dụng khoảng 18,8% số lượng tham số của MLP. Kết quả này cho thấy số lượng tham số không quyết định trực tiếp hiệu quả phân loại; cơ chế tích chập với khả năng khai thác đặc trưng cục bộ và chia sẻ trọng số giúp CNN học biểu diễn phù hợp hơn với cấu trúc không gian của ảnh.

Về chi phí, MiniVGG có thời gian huấn luyện mỗi epoch và bộ nhớ GPU đỉnh đạt mức cao nhất trong ba mô hình. Điều này phù hợp với việc mô hình phải xử lý các phép tích chập trên nhiều feature map, mặc dù tổng số tham số thấp hơn MLP.

3.3.2 Quá trình hội tụ

Hình 3.3 so sánh training loss và validation accuracy của ba mô hình qua 30 epoch. Mỗi đường biểu diễn trung bình trên ba seed (0, 1, 2); vùng tô mờ thể hiện ± độ lệch chuẩn mẫu.



Hình 3.3: Đường cong loss (trái) và accuracy (phải) trên tập huấn luyện và kiểm định của ba mô hình E1 qua 30 epoch. Các đường biểu diễn giá trị trung bình trên ba seed (0, 1, 2), với vùng tô thể hiện ± độ lệch chuẩn mẫu.

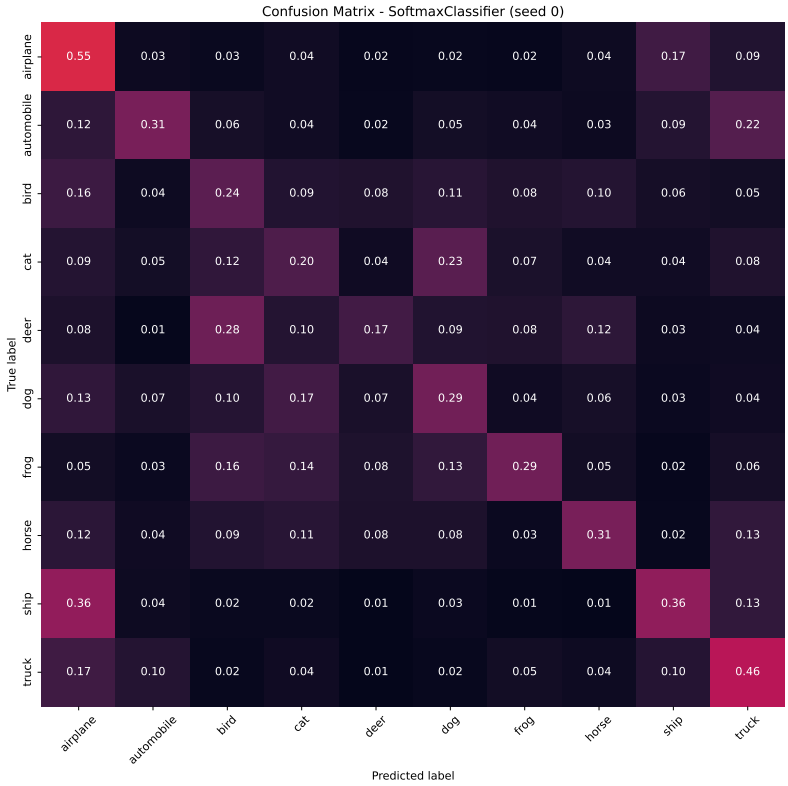
Softmax có training loss gần như không cải thiện sau các epoch đầu, duy trì quanh mức 2,09; validation accuracy dao động trong khoảng 29–35% mà không có xu hướng tăng rõ rệt. MLP hội tụ tốt hơn, với training loss giảm từ 1,83 xuống 1,42 và validation accuracy tăng từ 39,96% lên 50,11%, dù tốc độ cải thiện giảm dần về cuối.

MiniVGG cho thấy mức cải thiện lớn nhất: training loss giảm từ 1,44 xuống 0,61, đồng thời validation accuracy tăng từ 62,47% lên 80,79%. Đường cong tiếp tục cải thiện ở các epoch cuối, trong khi độ biến thiên giữa các seed tương đối nhỏ. Kết quả cho thấy MiniVGG duy trì khả năng học hiệu quả hơn hai kiến trúc sử dụng biểu diễn ảnh làm phẳng trong phạm vi 30 epoch thực nghiệm.

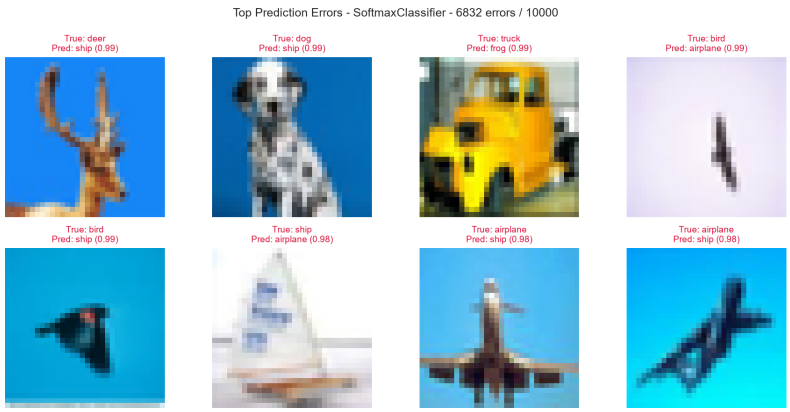
3.3.3 Phân tích lỗi phân loại

Để khảo sát những hạn chế của từng kiến trúc, nhóm phân tích ma trận nhầm lẫn và các ảnh dự đoán sai có độ tin cậy cao trên tập kiểm thử CIFAR-10. Phân tích sử dụng kết quả của seed 0; mỗi lớp có 1.000 ảnh kiểm thử.

3.3.3.1 Softmax



(a) Ma trận nhầm lẫn.



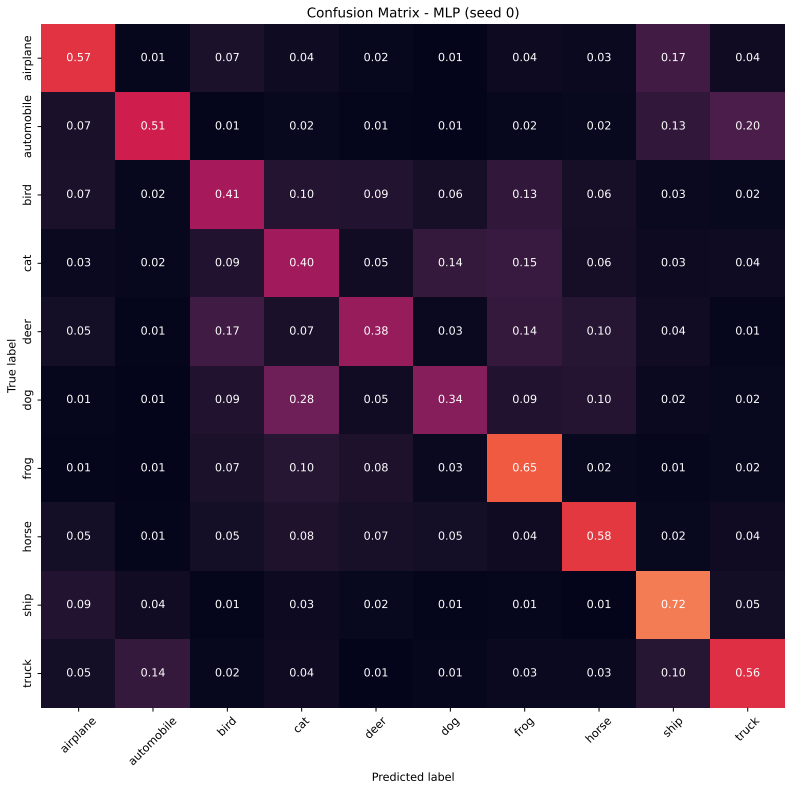
(b) Các ảnh dự đoán sai.

Hình 3.4: Phân tích lỗi của SoftmaxClassifier trên tập kiểm thử CIFAR-10 (seed 0).

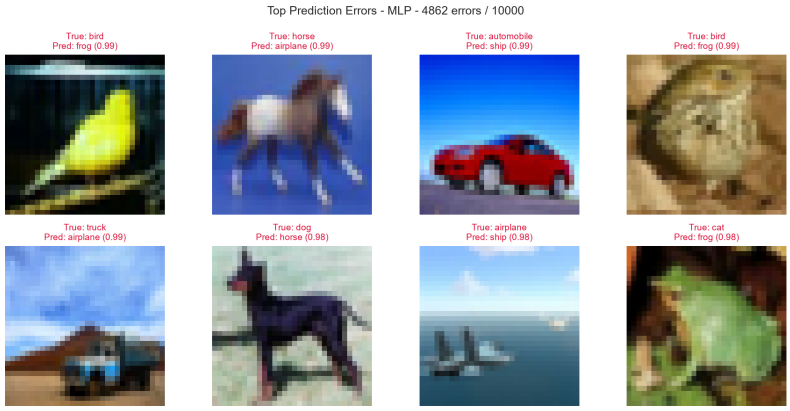
Với Softmax (seed 0, độ chính xác kiểm thử 31,68%), Hình 3.4 cho thấy các lớp được nhận diện tốt nhất là airplane (54,6%), truck (45,7%) và ship (36,0%); khó nhất là deer (17,2%), cat (20,4%) và bird (23,6%). Những nhầm lẫn nổi bật gồm ship→airplane (364 ảnh), deer→bird (280 ảnh), cat→dog (234 ảnh) và automobile→truck (220 ảnh). Các ảnh dự đoán sai có độ tin cậy cao còn xuất hiện những trường hợp deer hoặc dog bị nhận diện thành ship, với xác suất dự đoán

khoảng 0,99. Kết quả cho thấy hạn chế của mô hình tuyến tính khi biểu diễn các đặc trưng hình dạng và cấu trúc không gian của ảnh.

3.3.3.2 MLP



(a) Ma trận nhầm lẫn.



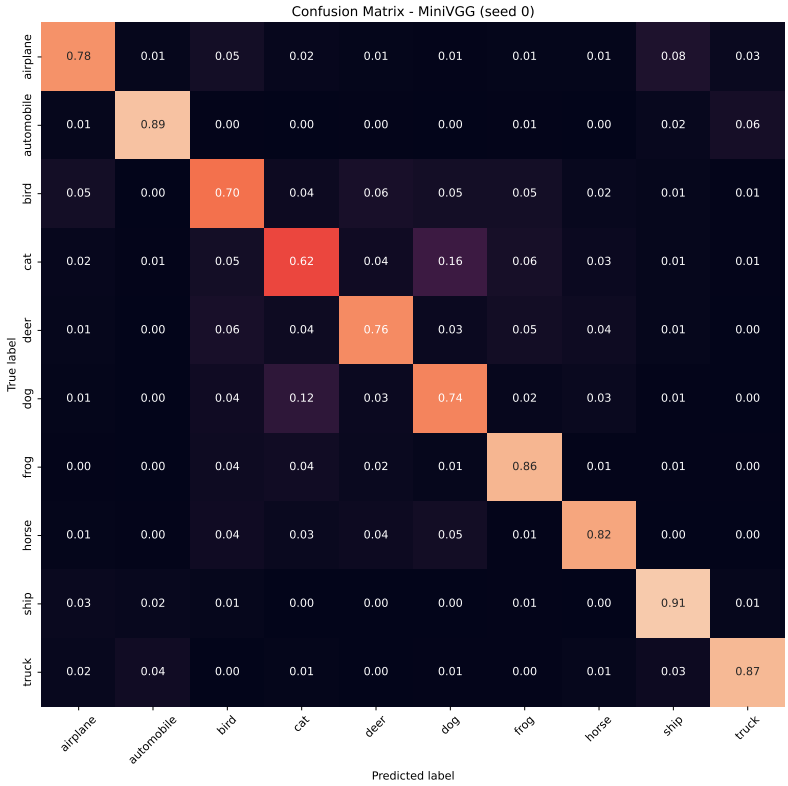
(b) Các ảnh dự đoán sai.

Hình 3.5: Phân tích lỗi của MLP trên tập kiểm thử CIFAR-10 (seed 0).

Với MLP (seed 0, độ chính xác kiểm thử 51,38%), Hình 3.5 cho thấy các lớp dễ nhận diện nhất là ship (72,5%), frog (65,3%) và horse (58,5%); khó nhất là dog (33,7%), deer (37,6%) và cat

(40,2%). Các cặp nhầm lẫn nổi bật gồm cat↔dog (140 và 283 ảnh), automobile↔truck (198 và 139 ảnh), airplane→ship (173 ảnh) và deer→bird (168 ảnh). Error grid ghi nhận những trường hợp bird bị dự đoán thành frog, automobile thành ship và horse thành airplane với xác suất khoảng 0,99. So với Softmax, MLP cải thiện khả năng nhận diện nhờ các biến đổi phi tuyến, nhưng vẫn gặp khó khăn với những lớp có đặc điểm hình ảnh tương đồng do chưa khai thác trực tiếp cấu trúc không gian của ảnh.

3.3.3.3 MiniVGG



(a) Ma trận nhầm lẫn.



(b) Các ảnh dự đoán sai.

Hình 3.6: Phân tích lỗi của MiniVGG trên tập kiểm thử CIFAR-10 (seed 0).

Với MiniVGG (seed 0, độ chính xác kiểm thử 79,52%), Hình 3.6 cho thấy các lớp dễ nhận diện nhất là ship (91,4%), automobile (89,1%), truck (86,8%) và frog (86,0%); khó nhất là cat (61,7%), bird (70,2%) và dog (74,4%). Các cặp nhầm lẫn nổi bật gồm cat↔dog (157 và 122 ảnh), bird↔deer (64 ảnh mỗi chiều), airplane→ship (81 ảnh) và automobile→truck (55 ảnh). Error grid cho thấy các trường hợp nhầm giữa automobile và truck, cũng như cat hoặc bird bị nhận diện

thành frog, với xác suất dự đoán hiển thị gần 1,00. So với hai kiến trúc còn lại, MiniVGG giảm đáng kể số lỗi phân loại nhờ khai thác các đặc trưng không gian, song vẫn gặp khó khăn ở những lớp động vật có hình dạng hoặc đặc điểm thị giác tương đồng.

3.4 Nhận xét và thảo luận

Trong thiết lập thực nghiệm này, đặc điểm kiến trúc có ảnh hưởng lớn hơn đến hiệu quả phân loại so với số lượng tham số. Softmax và MLP đều xử lý ảnh dưới dạng vector làm phẳng, trong đó MLP bổ sung các phép biến đổi phi tuyến nhưng không khai thác trực tiếp quan hệ không gian giữa các pixel. Ngược lại, MiniVGG sử dụng các phép tích chập và chia sẻ trọng số để học đặc trưng cục bộ. Mặc dù chỉ có khoảng 18,8% số tham số của MLP, MiniVGG đạt độ chính xác cao hơn đáng kể. Điều này cho thấy tính cục bộ của CNN phù hợp với bài toán phân loại ảnh hơn việc chỉ tăng số lượng tham số của mạng truyền thẳng.

Khả năng hội tụ và phân biệt các lớp. Softmax sớm chững lại, trong khi MLP và MiniVGG tiếp tục cải thiện trong quá trình huấn luyện. Phân tích lỗi cũng cho thấy các kiến trúc mạnh hơn giảm được nhiều trường hợp nhầm lẫn, nhưng những lớp có đặc trưng thị giác tương đồng, đặc biệt là cat và dog, vẫn khó phân biệt. Ngoài ra, cả ba mô hình đều xuất hiện những dự đoán sai với độ tin cậy cao, cho thấy accuracy tổng thể chưa phản ánh đầy đủ hành vi phân loại và mức độ đáng tin cậy của xác suất dự đoán.

Chi phí tính toán không tỉ lệ trực tiếp với số tham số. MiniVGG có ít tham số hơn MLP nhưng yêu cầu thời gian huấn luyện và bộ nhớ GPU lớn hơn. Nguyên nhân là các phép tích chập phải xử lý những bản đồ đặc trưng không gian qua nhiều lớp, tạo ra chi phí tính toán và bộ nhớ trung gian không được phản ánh đầy đủ qua số tham số. Vì vậy, việc lựa chọn kiến trúc cần xét đồng thời độ chính xác, thời gian xử lý và tài nguyên phần cứng.

Hạn chế. (i) Chỉ sử dụng ba seed, nên mức độ ổn định được đánh giá trong phạm vi số lần lặp còn hạn chế. (ii) Các mô hình dùng chung cấu hình huấn luyện, chưa tối ưu siêu tham số riêng cho từng kiến trúc. (iii) Thực nghiệm chỉ thực hiện trên CIFAR-10 trong 30 epoch, chưa đánh giá khả năng tổng quát hóa sang tập dữ liệu hoặc điều kiện huấn luyện khác. (iv) Thời gian và bộ nhớ được đo trên một cấu hình GPU, nên chưa phản ánh đầy đủ hiệu năng trên phần cứng khác. (v) Ma trận nhầm lẫn và các ảnh lỗi chỉ được minh họa bằng seed 0; các trường hợp lỗi được chọn không đại diện cho toàn bộ phân bố lỗi. (vi) Chưa thực hiện ablation study để đánh giá riêng ảnh hưởng của các kỹ thuật tăng cường dữ liệu (data augmentation) và các thành phần kiến trúc như Batch Normalization và pooling.

Chương 4

BÀI TẬP E2 – MULTI-HEAD SELF-ATTENTION VÀ TOKENIZATION

4.1 Kiến trúc và cài đặt mô hình

Bài tập E2 xây dựng bộ phân loại ảnh CIFAR-10 chỉ gồm các khối Transformer encoder, trong đó cơ chế Multi-Head Self-Attention (MSA) được hiện thực theo hai cách khác nhau nhưng nhận *cùng* trọng số khởi tạo: một bản tự viết từ các phép toán cơ bản và một bản bọc `nn.MultiheadAttention` của PyTorch. Nhờ đó, mọi khác biệt về kết quả giữa hai bản chỉ có thể đến từ cách hiện thực.

4.1.1 Multi-Head Self-Attention tự xây dựng

Cho chuỗi N token $X \in \mathbb{R}^{N\times D}$ với $D = 128$ và $h = 4$ head, mỗi head có chiều $d_h = D/h = 32$. Với mỗi head i, các phép chiếu tuyến tính cho $Q_i = XW_i^Q$, $K_i = XW_i^K$, $V_i = XW_i^V$ và

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}}\right) V_i, \qquad \text{MSA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O. \tag{4.1}$$

Hệ số $1/\sqrt{d_h}$ giữ phương sai của tích vô hướng ổn định theo d_h, tránh việc softmax bão hòa và gradient bị triệt tiêu [7].

Lớp `MSAScratch` chỉ sử dụng `nn.Linear`, `torch.einsum`, `softmax` và các phép `reshape`/`permute`. Ba phép chiếu Q, K, V được gộp thành một lớp tuyến tính $D \to 3D$ rồi tách head; tích $QK^\top$ và tích với V được viết bằng `einsum` trên chỉ số batch, head và token. Ma trận chú ý (mỗi hàng có tổng bằng 1) có thể được trả về để phục vụ trực quan hóa ở mục 4.4.4. Mỗi khối MSA có $4D^2 + 4D$ tham số (ba phép chiếu, phép chiếu đầu ra và các bias), tức 66 048 tham số với $D = 128$.

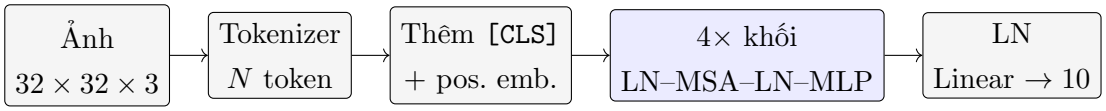
4.1.2 Multi-Head Self-Attention sử dụng PyTorch

Lớp `MSATorch` bọc `nn.MultiheadAttention` (`batch_first=True`) với cùng D, h và cùng bố cục tham số. Bộ trọng số của hai bản được sinh từ cùng một bộ sinh ngẫu nhiên theo seed (Xavier-uniform, bias bằng 0) và được nạp vào hai lớp theo cùng một quy ước; phần còn lại của mô hình (embedding, token `[CLS]`, positional embedding, MLP) cũng được khởi tạo sau khi đặt lại seed, nên hai mô hình bắt đầu huấn luyện từ *đúng cùng một điểm*. Cùng thứ tự dữ liệu và cùng augmentation theo seed được dùng cho cả hai bản.

Tính tương đương được kiểm tra bằng bộ kiểm thử đơn vị (`E2/tests`, 9 kiểm thử, tất cả đạt): với độ chính xác kép, đầu ra, ma trận chú ý và gradient của hai bản trùng nhau trong sai số 10^{-9}; số tham số bằng nhau; hai mô hình đầy đủ (tự viết và PyTorch) có cùng trạng thái ban đầu. Cần lưu ý rằng khi không cần ma trận chú ý, PyTorch có thể dùng các nhân tính toán gộp (fused kernel) nên thời gian chạy và bộ nhớ của hai bản không giống nhau, dù kết quả số học tương đương.

4.1.3 Mô hình phân loại ảnh sử dụng MSA

Mô hình là một Transformer encoder kiểu pre-norm (Hình 4.1): chuỗi token được tạo bởi một tokenizer (mục 4.2), thêm một token `[CLS]` học được ở đầu chuỗi và cộng positional embedding học được; sau đó qua $L = 4$ khối, mỗi khối gồm $x \leftarrow x + \mathrm{MSA}(\mathrm{LN}(x))$ và $x \leftarrow x + \mathrm{MLP}(\mathrm{LN}(x))$, MLP có chiều ẩn $4D = 512$, kích hoạt GELU và dropout 0,1. Đầu phân loại là một lớp tuyến tính áp dụng lên token `[CLS]` sau LayerNorm cuối. Mô hình có khoảng 0,81–0,90 triệu tham số tùy tokenizer (Bảng 4.4), được huấn luyện hoàn toàn từ đầu.



Hình 4.1: Sơ đồ mô hình phân loại ảnh dùng MSA. Chỉ khối MSA (tô xanh) được hiện thực hai cách; các phần còn lại giống hệt nhau.

4.2 Các phương pháp Tokenization

Transformer xử lý chuỗi token 1D nên ảnh 32×32 cần được chuyển thành chuỗi vector D chiều. Bài tập triển khai bốn cách lấy token (Bảng 4.1); mọi mô hình đều thêm token `[CLS]`, nên độ dài chuỗi đưa vào attention là $N + 1$.

Tokenizer	N	$N{+}1$	Cách tạo token
Patch 4×4	64	65	Chia ảnh thành các ô 4×4 không chồng lấn, trải phẳng (48 giá trị) và chiếu tuyến tính lên D (tương đương `Conv2d` có kernel = stride = 4).
Patch 8×8	16	17	Tương tự với ô 8×8 (192 giá trị mỗi token).
Hàng (row)	32	33	Mỗi hàng ảnh ($3 \times 32 = 96$ giá trị) là một token; cấu trúc 2D chỉ còn lại ở thứ tự hàng.
CNN-stem	64	65	Ba lớp tích chập 3×3 (stride 1, 2, 2; BatchNorm và GELU) tạo bản đồ đặc trưng 8×8; mỗi vị trí là một token.

Bảng 4.1: Bốn cách tokenization được triển khai.

Patch 4×4 và 8×8 thuộc kiểu ViT [2], cho phép so sánh ảnh hưởng của số token N (64 so với 16) lên độ chính xác và chi phí tính toán, vì chi phí attention tăng theo N^2. Tokenizer theo hàng là trường hợp token gần như không giữ tính cục bộ 2D. CNN-stem là kiến trúc lai: tương tác cục bộ được học bằng tích chập trước khi vào attention. Các biến thể khác nêu trong đề (patch chồng lấn, token theo kênh, token theo từng pixel với $N = 1024$) không được thực hiện: ba mức $N = 16, 32, 64$ đã đủ để quan sát tác động của độ dài chuỗi, trong khi $N = 1024$ đòi hỏi bộ nhớ và thời gian vượt quá một máy GPU 6 GB trong phạm vi bài tập.

4.3 Huấn luyện và đánh giá

4.3.1 Quy trình huấn luyện

Mọi mô hình E2 dùng cùng dữ liệu và cùng quy trình (dữ liệu, phép chuẩn hóa và augmentation theo mục 2.1–2.5). Cấu hình huấn luyện được tóm tắt ở Bảng 4.2; các siêu tham số được chọn theo công thức phổ biến cho Transformer nhỏ huấn luyện từ đầu và được *cố định cho mọi mô hình* (không tinh chỉnh riêng từng tokenizer trên tập kiểm thử).

Thành phần	Thiết lập
Kiến trúc	Pre-norm, $D = 128$, 4 khối, 4 head, MLP 512, dropout 0,1
Tối ưu	AdamW, lr đỉnh 10^{-3}, weight decay 0,05, cắt gradient 1,0
Lịch lr	Warmup tuyến tính 5% số bước, sau đó cosine về 1% lr đỉnh
Hàm mất mát	Cross-entropy, label smoothing 0,1
Huấn luyện	30 epoch, batch 128, độ chính xác hỗn hợp fp16
Augmentation	Random crop (đệm 4) và lật ngang, chỉ trên tập huấn luyện
Seed	$0, 1, 2$ (ảnh hưởng khởi tạo, thứ tự dữ liệu và augmentation)

Bảng 4.2: Cấu hình huấn luyện của E2.

4.3.2 Các thí nghiệm

Tổng cộng có 30 lần huấn luyện: 4 tokenizer $\times$ 2 bản MSA (tự viết, PyTorch) $\times$ 3 seed $= 24$ lần cho hai so sánh chính (tương đương hiện thực và so sánh tokenizer), cộng 2 tokenizer (Patch 4×4, Hàng) $\times$ 3 seed $= 6$ lần *không có* positional embedding cho thí nghiệm bóc tách (ablation).

4.3.3 Tiêu chí đánh giá

Chỉ số chính là độ chính xác trên 10 000 ảnh kiểm thử chính thức, tính bằng trọng số của epoch *cuối cùng* (không chọn checkpoint theo tập kiểm thử), báo cáo dưới dạng trung bình $\pm$ độ lệch chuẩn mẫu trên 3 seed. Độ chính xác validation chỉ dùng để theo dõi. Ngoài ra báo cáo số tham số, số phép nhân-cộng (MACs) cho một ảnh, thời gian huấn luyện trung vị mỗi epoch và bộ nhớ GPU đỉnh. Hai bản MSA được so sánh theo cặp seed (cùng khởi tạo, cùng thứ tự dữ liệu). Với $n = 3$ seed, các chênh lệch nhỏ hơn độ lệch chuẩn giữa các seed không được xem là có ý nghĩa.

4.3.4 Môi trường thực nghiệm

Mọi lần chạy E2 thực hiện cục bộ, một lần liền mạch (không gián đoạn), khoảng 2,5 giờ, trên máy laptop cắm điện ở chế độ hiệu năng cao, không chạy ứng dụng nặng khác: CPU AMD Ryzen 5 6600H, RAM 16 GB, GPU NVIDIA GeForce RTX 3050 Laptop 6 GB, Python 3.12, PyTorch 2.14.1 (CUDA 12.6). Thời gian đo được chỉ có giá trị so sánh *tương đối* giữa các mô hình trên cùng máy.

4.4 Kết quả thực nghiệm

Mọi số liệu dưới đây là trung bình ± độ lệch chuẩn mẫu trên ba seed $(0, 1, 2)$, độ chính xác tính bằng % trên tập kiểm thử chính thức ở epoch cuối, trừ khi ghi chú khác. Số liệu thô (tệp JSON) nằm trong thư mục `E2/results`; các bảng được sinh tự động bởi `python -m e2lib.analyze`.

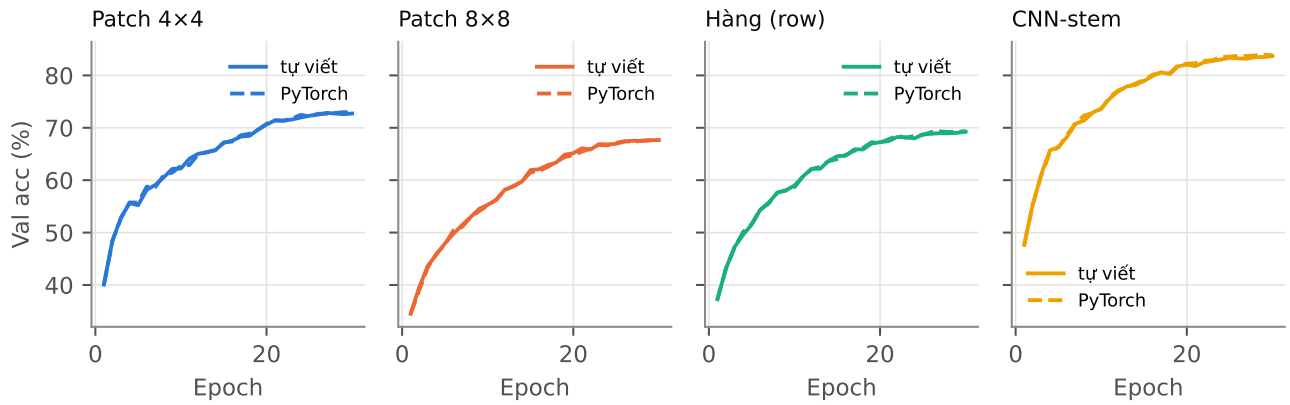
4.4.1 MSA tự viết và MSA của PyTorch

Bảng 4.3 so sánh hai bản hiện thực với cùng tokenizer. Δ là chênh lệch trung bình theo cặp seed (tự viết − PyTorch), tính bằng điểm phần trăm.

Tokenizer	Tham số	Tự viết	PyTorch	Δ	s/ep. (tự viết)	s/ep. (PyTorch)
Patch 4×4	809 354	72,42 ± 0,86	72,41 ± 1,03	+0,02	9,1	8,6
Patch 8×8	821 642	67,09 ± 0,84	67,04 ± 0,83	+0,05	7,0	5,8
Hàng (row)	811 402	68,96 ± 0,41	68,92 ± 0,16	+0,04	6,8	5,7
CNN-stem	896 426	83,00 ± 0,41	83,19 ± 0,54	−0,18	10,3	9,9

Bảng 4.3: Độ chính xác kiểm thử (%) và thời gian huấn luyện mỗi epoch (s/ep.) của hai bản MSA.

Chênh lệch trung bình chỉ từ −0,18 đến +0,05 điểm, nhỏ hơn nhiều so với độ lệch chuẩn giữa các seed (0,16–1,03 điểm) và không có dấu nhất quán. Theo từng seed, chênh lệch lớn nhất là 0,68 điểm (CNN-stem, seed 1); ở Patch 4×4 seed 0 hai bản cho cùng độ chính xác. Các khác biệt còn lại đến từ nhiễu số học (thứ tự phép cộng dấu phẩy động, nhân tính toán khác nhau, fp16) tích lũy qua 30 epoch, không phải từ khác biệt về thuật toán. Đường cong validation của hai bản gần như chồng khít (Hình 4.2). Về chi phí, bản PyTorch nhanh hơn khoảng 4% (Patch 4×4, CNN-stem) đến 16–17% (Patch 8×8, Hàng) mỗi epoch và dùng ít bộ nhớ đỉnh hơn (ví dụ 394 so với 450 MB ở Patch 4×4), nhờ nhân tính toán gộp; khi các phần khác của mô hình chiếm phần lớn thời gian (CNN-stem, nhiều token) lợi thế này nhỏ hơn.



Hình 4.2: Độ chính xác validation theo epoch của MSA tự viết (nét liền) và PyTorch (nét đứt), trung bình ba seed.

4.4.2 So sánh các phương pháp tokenization

Bảng 4.4 so sánh bốn tokenizer với MSA tự viết; Bảng 4.5 bổ sung chi phí tính toán lý thuyết và đo được.

Tokenizer	$N+1$	Tham số	Val	Test	s/epoch	Bộ nhớ (MB)
Patch 4×4	65	809 354	72,73 ± 0,91	72,42 ± 0,86	9,1	450
Patch 8×8	17	821 642	67,65 ± 0,85	67,09 ± 0,84	7,0	265
Hàng (row)	33	811 402	69,25 ± 0,74	68,96 ± 0,41	6,8	323
CNN-stem	65	896 426	83,69 ± 0,32	83,00 ± 0,41	10,3	487

Bảng 4.4: So sánh bốn tokenizer (MSA tự viết). Độ chính xác validation và kiểm thử (%); thời gian là trung vị giây/epoch huấn luyện; bộ nhớ là đỉnh GPU khi huấn luyện.

Tokenizer	$N+1$	MACs / ảnh (triệu)	ms / bước (batch 128)
Patch 4×4	65	55,8	24,8
Patch 8×8	17	14,1	17,4
Hàng (row)	33	27,5	17,5
CNN-stem	65	65,8	28,0

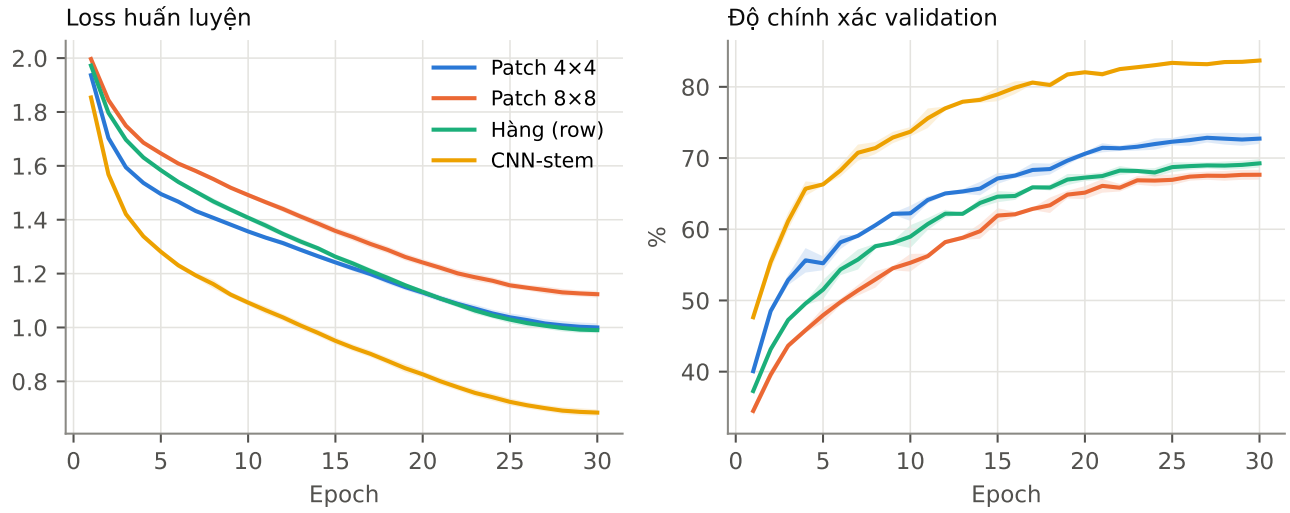
Bảng 4.5: Chi phí tính toán một lượt thuận (MSA tự viết). MACs đếm các phép nhân ma trận (Linear, $QK^\top$, AV, tích chập) và khớp với công thức giải tích; ms/bước đo huấn luyện đầy đủ trên RTX 3050 Laptop.

Về số MACs: công cụ đếm FLOP tự động bỏ sót toàn bộ khối MSA khi chạy bản PyTorch (nhân gộp), nên con số của bản đó thấp hơn (ví dụ 34,5 so với 55,8 triệu ở Patch 4×4). Hai bản thực hiện cùng phép toán, vì vậy bảng dùng số của bản tự viết; số này khớp công thức $L\,(12ND^2 + 2N^2D)$ cộng chi phí embedding (55,84 triệu ở Patch 4×4 tính tay).

Kết quả chính:

- **CNN-stem tốt nhất rõ rệt:** $83{,}00 \pm 0{,}41\%$, hơn Patch 4×4 ($72{,}42 \pm 0{,}86\%$) khoảng 10,6 điểm, trong khi chi phí chỉ tăng khoảng 13% thời gian mỗi epoch. Khoảng cách lớn hơn rất nhiều so với dao động giữa các seed.
- **Patch 4×4 hơn Patch 8×8** (72,42 so với 67,09%; +5,3 điểm) nhưng đắt hơn: số token gấp $\approx 3{,}8$ lần, MACs gấp ≈ 4 lần, thời gian mỗi epoch khoảng 1,3 lần và bộ nhớ gần 1,7 lần.
- **Hàng** (68,96%) nằm giữa hai bản patch; nó cao hơn Patch 8×8 ở cả ba seed (thấp nhất 68,5% so với cao nhất 67,9%), nhưng chênh lệch $\approx 1{,}9$ điểm với $n = 3$ chỉ nên xem là gợi ý.
- Thứ hạng theo kiểm thử: CNN-stem > Patch 4×4 > Hàng > Patch 8×8; thứ hạng theo validation giống hệt.

Hình 4.3 cho thấy CNN-stem hội tụ nhanh hơn và đạt loss huấn luyện thấp hơn hẳn. Ở epoch cuối, độ chính xác huấn luyện (đo trên các batch đã augment) là $\approx$ 71–78% với các mô hình patch/hàng so với 92% của CNN-stem, trong khi khoảng cách huấn luyện–kiểm thử của cả bốn tokenizer đều nằm trong khoảng 4,6–9,1 điểm. Như vậy các mô hình patch/hàng chủ yếu bị hạn chế bởi khả năng khớp dữ liệu trong 30 epoch chứ không phải bởi quá khớp.



Hình 4.3: Loss huấn luyện (trái) và độ chính xác validation (phải) theo epoch của bốn tokenizer; đường là trung bình, vùng mờ là ± std ba seed.

4.4.3 Vai trò của positional embedding

Bảng 4.6 so sánh mô hình có và không có positional embedding (bản không có dùng vector không và cố định; MSA tự viết).

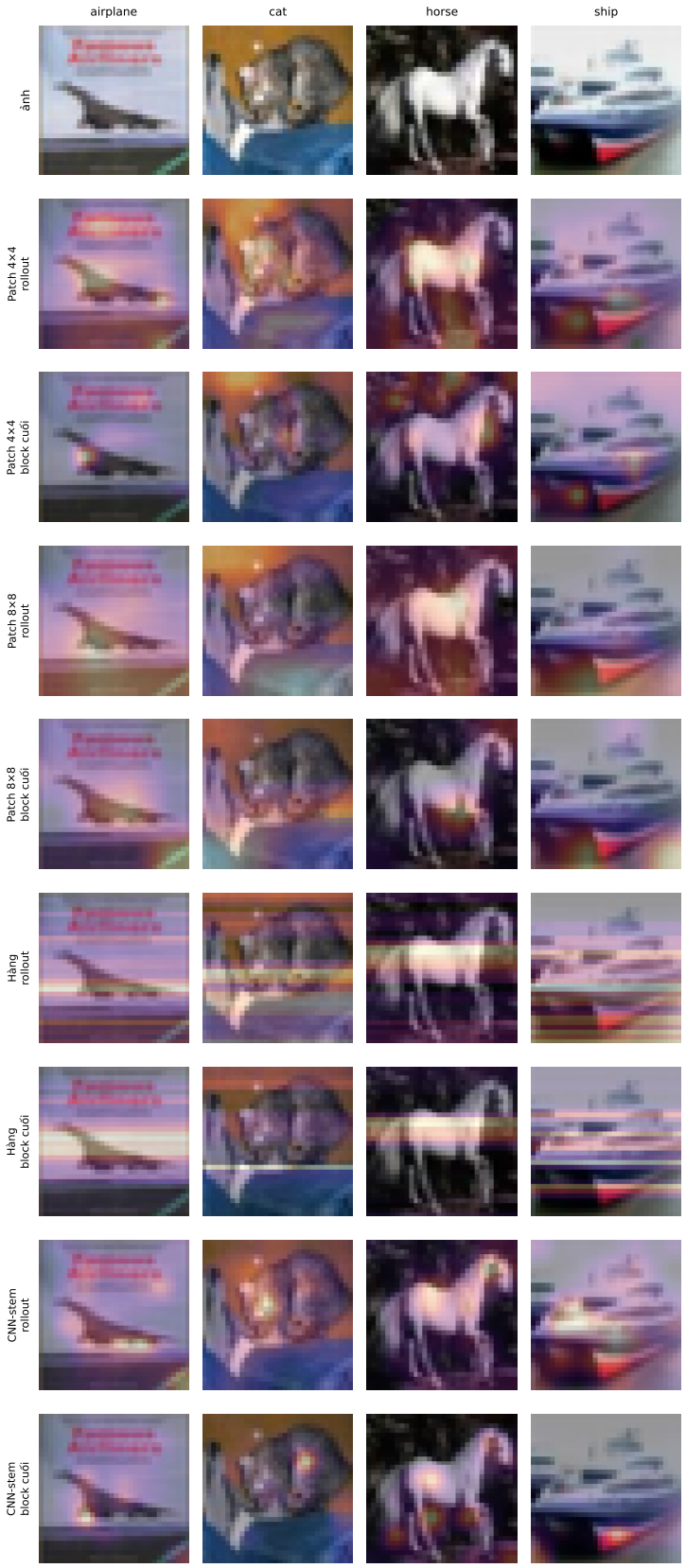
Tokenizer	**Có pos. emb.**	**Không có**	**Δ** (điểm)
Patch 4×4	72,42 ± 0,86	64,78 ± 0,31	−7,65
Hàng (row)	68,96 ± 0,41	62,20 ± 0,65	−6,76

Bảng 4.6: Ablation positional embedding: độ chính xác kiểm thử (%).

Bỏ positional embedding làm giảm độ chính xác 7,65 điểm ở Patch 4×4 và 6,76 điểm ở Hàng, lớn hơn nhiều so với nhiễu giữa các seed ($\leq$ 1,03). Độ chính xác *huấn luyện* cũng giảm ($\approx 77\% \rightarrow 71\%$ ở Patch 4×4), tức mô hình mất khả năng biểu diễn chứ không chỉ khái quát kém hơn. Điều này phù hợp với việc self-attention không phụ thuộc thứ tự token: không có thông tin vị trí, mô hình chỉ thấy một *tập* patch/hàng. Mô hình vẫn đạt 62–65%, vì nội dung từng token (màu, kết cấu, đặc điểm của hàng) đã mang nhiều thông tin về lớp. Thí nghiệm này không chạy cho CNN-stem, nơi các lớp tích chập có thể truyền một phần thông tin vị trí vào token.

4.4.4 Trực quan hóa bản đồ chú ý

Hình 4.4 vẽ, cho bốn ảnh kiểm thử, *attention rollout* [1] (nhân các ma trận chú ý trung bình theo head, đã cộng đường residual, qua các khối) và chú ý của `[CLS]` ở khối cuối, với mô hình tự viết seed 0. Các bản đồ chỉ mang tính gợi ý, không phải bằng chứng nhân quả về căn cứ quyết định của mô hình.

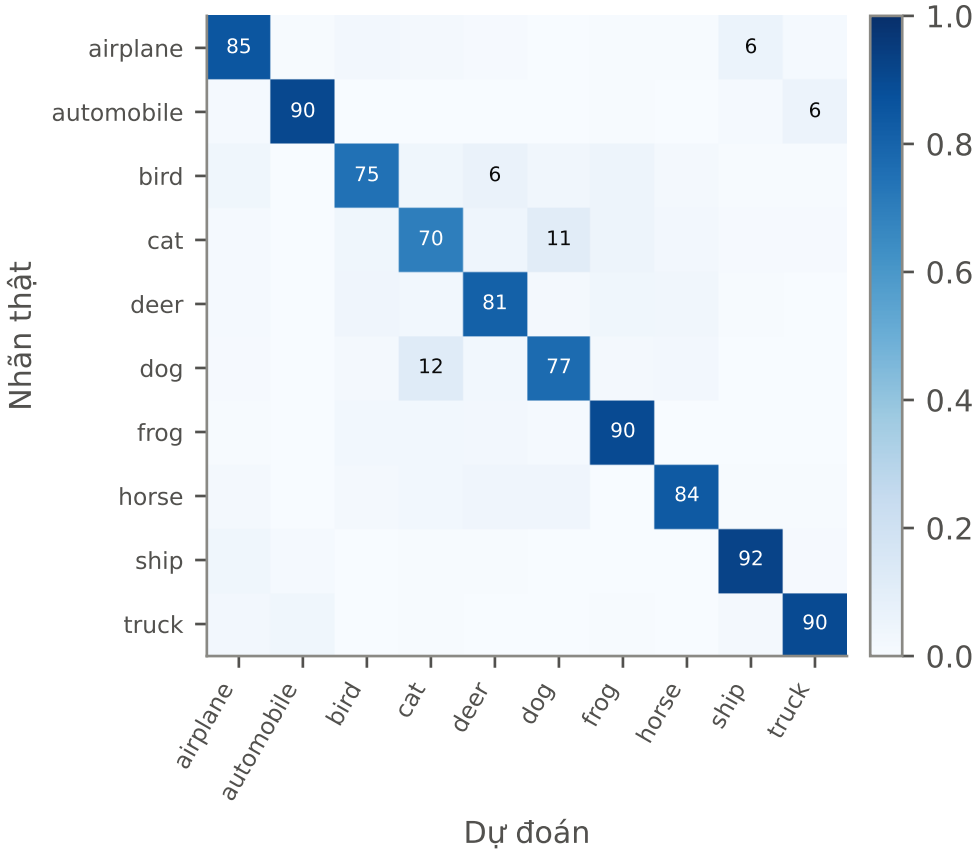


Hình 4.4: Chú ý của `[CLS]` lên ảnh đầu vào: rollout và khối cuối cho từng tokenizer (seed 0).

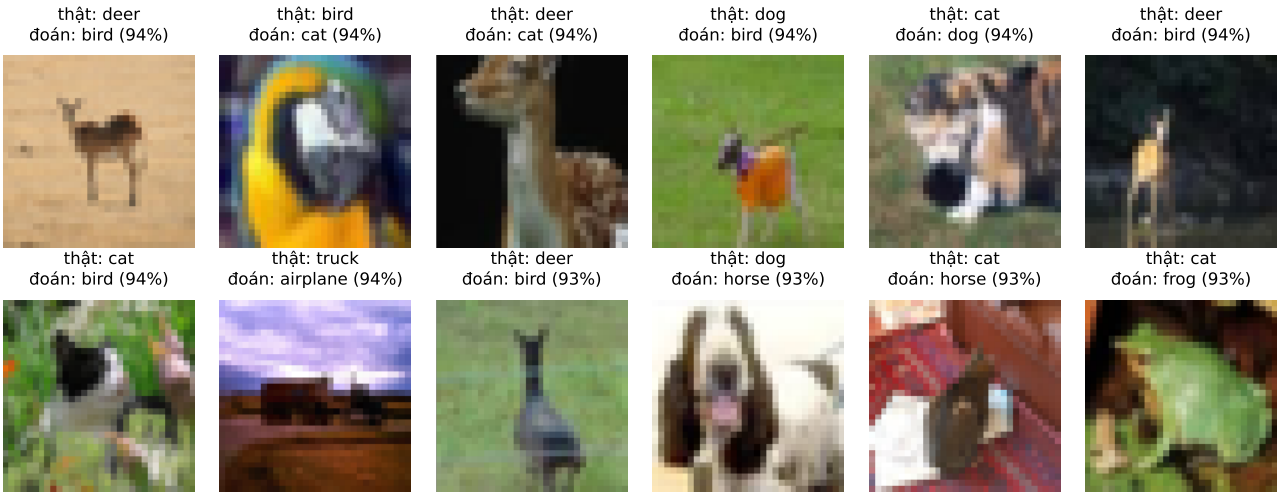
Quan sát: (i) với Patch 4×4 và CNN-stem, rollout tập trung khá rõ vào thân con ngựa và vùng quanh con tàu và máy bay, còn chú ý khối cuối thường dồn vào một vài điểm cục bộ (đầu, chân, mũi tàu) và đôi khi rơi vào nền; (ii) với Patch 8×8 vùng chú ý thô hơn do chỉ có 16 token; (iii) với tokenizer theo hàng, bản đồ là các dải ngang vì mỗi token là cả một hàng; mô hình ưu tiên một số hàng nhất định (ví dụ hàng giữa ở ảnh con ngựa) thay vì vật thể theo hình dạng. Với bốn ảnh và một seed, các nhận xét này mang tính minh họa, không được khái quát hóa.

4.4.5 Phân tích lỗi

Với mô hình tốt nhất (CNN-stem, seed 0, kiểm thử 83,34%), Hình 4.5 là ma trận nhầm lẫn chuẩn hóa theo hàng. Lớp dễ nhất là *ship* (92,2%), *automobile* (90,4%), *frog* (89,6%) và *truck* (89,5%); khó nhất là *cat* (69,8%), *bird* (74,8%) và *dog* (77,2%). Các cặp nhầm nhiều nhất là cat↔dog (111 và 120 ảnh), bird→deer (63), airplane→ship (59) và automobile→truck (58), tức các lớp có hình dạng, màu sắc hoặc bối cảnh tương tự; đây cũng là các cặp được dự đoán trước ở mục phân tích dữ liệu. Hình 4.6 cho thấy các ảnh bị dự đoán sai với độ tin cậy cao nhất: nhiều ảnh có vật thể nhỏ, bị che hoặc khó nhận biết ở độ phân giải 32×32.



Hình 4.5: Ma trận nhầm lẫn (CNN-stem, seed 0; % theo hàng).



Hình 4.6: Các ảnh dự đoán sai với độ tin cậy cao nhất.

4.5 Nhận xét và thảo luận

Hiện thực MSA. Bản MSA tự viết (chỉ `Linear`, `einsum`, `softmax`) cho kết quả tương đương bản `nn.MultiheadAttention`: kiểm thử đơn vị xác nhận đầu ra, ma trận chú ý và gradient trùng nhau, và sau 30 epoch chênh lệch độ chính xác nằm trong nhiễu giữa các seed. Lợi ích của bản PyTorch chỉ là tốc độ và bộ nhớ (nhanh hơn 4–17% mỗi epoch trong thiết lập này).

Tokenization quan trọng hơn kích thước attention. Với cùng encoder gần 0,8–0,9 triệu tham số, thay cách tạo token làm độ chính xác thay đổi từ 67% đến 83%. Tokenizer lai CNN-stem vượt xa các tokenizer tuyến tính, phù hợp với việc dữ liệu nhỏ (45 000 ảnh) và huấn luyện từ đầu thiếu thiên kiến quy nạp (inductive bias) về tính cục bộ mà tích chập cung cấp sẵn; điều này nhất quán với quan sát chung rằng ViT cần nhiều dữ liệu hoặc tiền huấn luyện để vượt CNN [2]. Giữa các tokenizer thuần attention, token nhỏ hơn (Patch 4×4) tốt hơn token lớn (8×8) nhưng chi phí tăng nhanh theo N^2; không có tokenizer nào vừa rẻ vừa chính xác nhất. Tokenizer theo hàng cho kết quả tốt đáng ngạc nhiên so với việc nó bỏ hầu hết cấu trúc 2D, nhưng vẫn kém Patch 4×4.

Thông tin vị trí là cần thiết. Bỏ positional embedding làm giảm ≈ 7 điểm ở cả hai tokenizer được thử, cho thấy thứ tự token mang thông tin không thể thay thế bằng nội dung token.

Hạn chế. (i) Chỉ 3 seed, nên các chênh lệch dưới ≈ 1 điểm (ví dụ Hàng so với Patch 8×8) chưa đủ chắc chắn. (ii) Siêu tham số được giữ cố định cho mọi tokenizer, không tinh chỉnh riêng; một tokenizer có thể hưởng lợi từ lịch huấn luyện khác. (iii) Mô hình nhỏ và chỉ 30 epoch; CNN-stem có độ chính xác huấn luyện cao hơn hẳn và có thể còn cải thiện, trong khi các mô hình khác còn hội tụ chưa hoàn toàn (đường cong vẫn đi lên nhẹ ở epoch cuối). (iv) Thời gian và bộ nhớ chỉ đo trên một GPU xách tay, một lần chạy mỗi cấu hình. (v) CNN-stem có nhiều tham số hơn khoảng

10% (không cân bằng hoàn toàn số tham số). (vi) Bản đồ chú ý chỉ minh họa trên bốn ảnh và một seed. (vii) Các biến thể token trong đề (patch chồng lấn, token theo kênh, từng pixel) chưa được thử.

Chương 5

BÀI TẬP E3 – PHÂN LOẠI ẢNH VỚI LSTM VÀ GRU

5.1 Các phương pháp Tokenization

LSTM và GRU xử lý dữ liệu dạng chuỗi nên ảnh CIFAR-10 kích thước 32×32 cần được chuyển thành chuỗi vector đặc trưng. Bài tập triển khai hai phương pháp tokenization (Bảng 5.1), với số chiều đặc trưng $D = 64$.

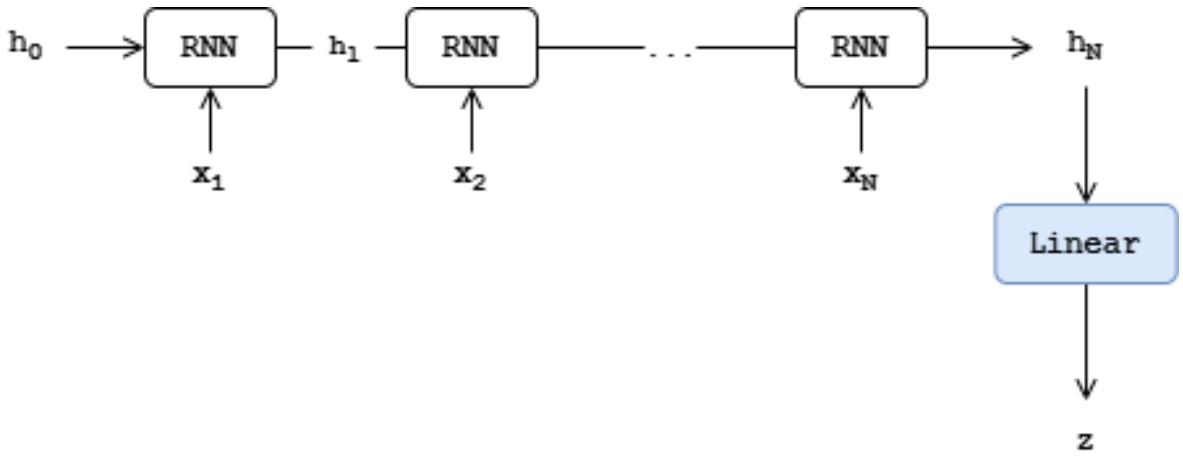
Tokenizer	N	**Đầu vào**	**Cách tạo token**
Hàng (Row)	32	$(B, 32, 64)$	Mỗi hàng ảnh gồm $3 \times 32 = 96$ giá trị, được chiếu tuyến tính lên 64 chiều.
Patch 4×4	64	$(B, 64, 64)$	Chia ảnh thành 64 vùng không chồng lấn, mỗi vùng gồm $3 \times 4 \times 4 = 48$ giá trị. Ánh xạ bằng `Conv2d` với `kernel_size = stride = 4`.

Bảng 5.1: Hai phương pháp tokenization được triển khai.

Row Tokenization xử lý các hàng ảnh tuần tự từ trên xuống dưới, tạo thành chuỗi 32 token. Đối với Patch4 Tokenization, ảnh được chia thành lưới 8×8 patch và các token được sắp xếp theo thứ tự hàng (*row-major order*): từ trái sang phải trên từng hàng patch, sau đó lần lượt từ trên xuống dưới. Cả hai phương pháp đều giữ nguyên thứ tự token trong quá trình huấn luyện và đánh giá. Do LSTM và GRU xử lý chuỗi theo thứ tự, cách sắp xếp này xác định trình tự tiếp nhận thông tin ảnh của mô hình. Row Tokenization tạo chuỗi ngắn hơn nhưng mỗi token bao phủ toàn bộ chiều rộng ảnh. Ngược lại, Patch Tokenization biểu diễn ảnh theo các vùng cục bộ với số bước chuỗi lớn hơn.

5.2 Kiến trúc các mô hình

Bài tập triển khai hai mô hình LSTM và GRU theo hướng *many-to-one* (Hình 5.1). Ảnh CIFAR-10 được chuyển thành chuỗi gồm N token thông qua Row hoặc Patch4 Tokenization. Tại mỗi bước, recurrent cell tiếp nhận token hiện tại và hidden statetừ bước trước để cập nhật hidden state mới.



Hình 5.1: Kiến trúc phân loại *many-to-one* sử dụng hidden state cuối cùng.

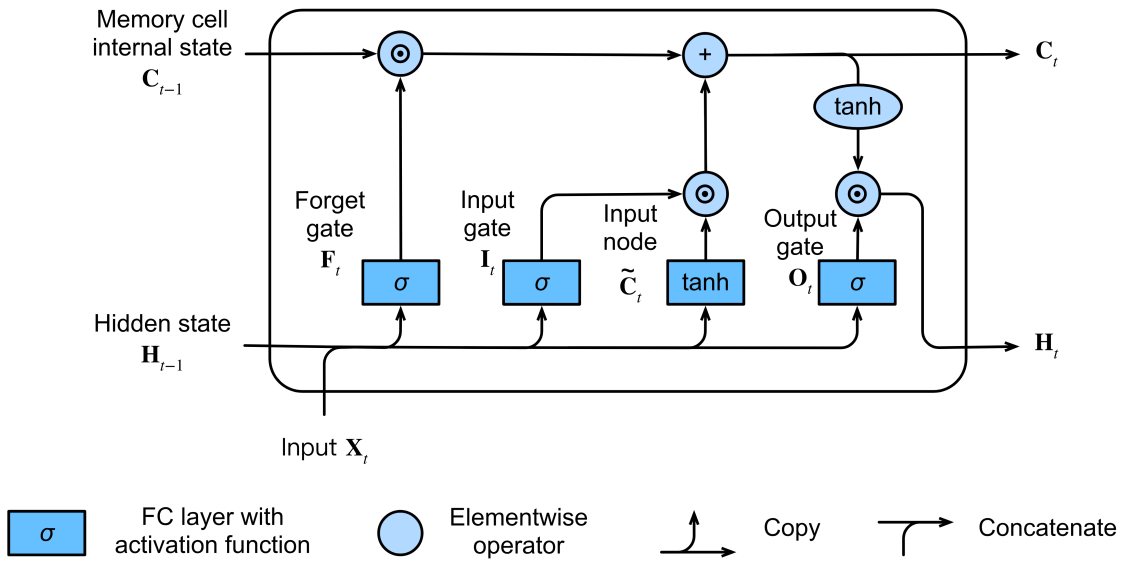
Cả hai mô hình sử dụng cùng embedding dimension $D = 64$, hidden size $H = 128$ và một recurrent layer đơn hướng. Sau khi xử lý toàn bộ chuỗi, hidden state cuối cùng $\mathbf{h}_N$ được đưa qua một lớp Linear để tạo logits cho 10 lớp CIFAR-10:

$$\mathbf{z} = W_c\mathbf{h}_N + \mathbf{b}_c, \qquad \mathbf{z} \in \mathbb{R}^{10}. \tag{5.1}$$

Hai mô hình có cùng kiến trúc tổng thể và cấu hình, chỉ khác nhau ở recurrent cell: LSTM sử dụng ba cổng điều khiển và cell state riêng biệt, trong khi GRU sử dụng hai cổng để cập nhật trực tiếp hidden state.

5.2.1 Mô hình LSTM

Long Short-Term Memory (LSTM) là một biến thể của mạng nơ-ron hồi quy (RNN), được thiết kế nhằm cải thiện khả năng học các quan hệ phụ thuộc dài hạn trong dữ liệu chuỗi. Thay vì chỉ duy trì trạng thái ẩn như RNN truyền thống, LSTM sử dụng hai trạng thái: trạng thái ẩn $\mathbf{h}_t$ và trạng thái bộ nhớ $\mathbf{c}_t$, kết hợp với các cơ chế cổng để điều khiển luồng thông tin.



Hình 5.2: Cấu trúc bên trong một ô LSTM. Nguồn: Dive into Deep Learning [9].

Như minh họa ở Hình 5.2, một ô LSTM sử dụng ba cổng chính: *forget gate* ($\mathbf{f}_t$) quyết định mức độ giữ lại thông tin từ trạng thái bộ nhớ trước đó; *input gate* ($\mathbf{i}_t$) kiểm soát việc bổ sung thông tin mới; và *output gate* ($\mathbf{o}_t$) điều khiển thông tin được đưa ra trạng thái ẩn. Trạng thái bộ nhớ và trạng thái ẩn được cập nhật theo:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \tag{5.2}$$

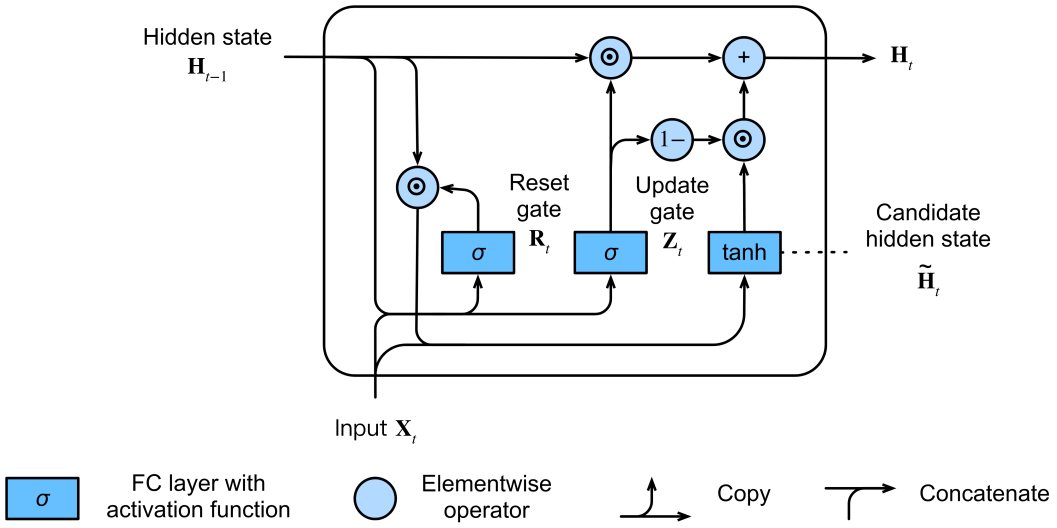
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \tag{5.3}$$

trong đó $\mathbf{g}_t$ là nội dung bộ nhớ ứng viên.

Trong bài tập, nhóm sử dụng `nn.LSTM` của PyTorch với một lớp (`num_layers=1`) hồi quy đơn hướng (`bidirectional=False`), kích thước đầu vào $D = 64$ (`input_size=64`) và kích thước trạng thái ẩn $H = 128$ (`hidden_size=128`).

5.2.2 Mô hình GRU

Gated Recurrent Unit (GRU) là một biến thể khác của mạng nơ-ron hồi quy (RNN). So với LSTM, GRU có cấu trúc đơn giản hơn do chỉ duy trì hidden state $\mathbf{h}_t$, không sử dụng cell state riêng biệt.



Hình 5.3: Cấu trúc GRU cell. Nguồn: Dive into Deep Learning [8].

Như minh họa ở Hình 5.3, GRU cell sử dụng hai cổng điều khiển: *reset gate* ($\mathbf{r}_t$) kiểm soát mức độ ảnh hưởng của hidden state trước đó khi tạo hidden state ứng viên; và *update gate* ($\mathbf{z}_t$) quyết định mức độ giữ lại thông tin cũ so với thông tin mới.

Hidden state tại bước t được cập nhật theo:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}, \tag{5.4}$$

trong đó $\tilde{\mathbf{h}}_t$ là hidden state ứng viên.

Trong bài tập, nhóm sử dụng `nn.GRU` của PyTorch với một lớp (`num_layers=1`) hồi quy đơn hướng (`bidirectional=False`), kích thước đầu vào $D = 64$ (`input_size=64`) và kích thước trạng thái ẩn $H = 128$ (`hidden_size=128`).

5.3 Huấn luyện và đánh giá

5.3.1 Quy trình huấn luyện

Các mô hình LSTM và GRU được huấn luyện trên cùng tập CIFAR-10, sử dụng chung phép chia dữ liệu, chuẩn hóa và augmentation đã trình bày ở mục 2.1–2.5. Các siêu tham số được giữ cố định cho cả bốn cấu hình nhằm đảm bảo tính nhất quán khi so sánh kiến trúc hồi quy và phương pháp tokenization.

Cấu hình huấn luyện được trình bày trong Bảng 5.2.

Thành phần	Thiết lập
Kiến trúc	1 recurrent layer đơn hướng, $D = 64$, $H = 128$
Tối ưu	Adam, learning rate cố định 10^{-3}, không weight decay
Hàm mất mát	Cross-Entropy
Gradient clipping	Global norm 1.0
Huấn luyện	30 epoch, batch size 64, fp32
Augmentation	Random crop (padding 4) và horizontal flip, chỉ áp dụng trên tập train
Random seeds	0, 1, 2

Bảng 5.2: Cấu hình huấn luyện của E3.

5.3.2 Các thí nghiệm

So sánh các cấu hình LSTM/GRU. Nhóm triển khai bốn cấu hình gồm LSTM-Row, LSTM-Patch4,GRU-Row và GRU-Patch4. Mỗi cấu hình được huấn luyện với ba random seeds, tổng cộng 12 lần huấn luyện (2 tokenizer $\times$ 2 kiến trúc $\times$ 3 seed).

So sánh với baseline MLP/CNN. Nhóm sử dụng lại checkpoint của hai mô hình MLP và MiniVGG đã huấn luyện trong E1, không huấn luyện lại. Các checkpoint được tải để đánh giá trên cùng tập test CIFAR-10. Kết quả được sử dụng làm baseline không hồi quy để so sánh accuracy và số tham số với các mô hình LSTM/GRU, đồng thời thảo luận sự khác biệt trong cách khai thác thông tin giữa các kiến trúc.

5.3.3 Tiêu chí đánh giá

So sánh các cấu hình LSTM/GRU.

Chỉ số chính là độ chính xác trên 10.000 ảnh test, được tính bằng trọng số tại epoch thứ 30 (epoch cuối cùng). Kết quả được báo cáo dưới dạng trung bình $\pm$ độ lệch chuẩn mẫu trên 3 seed. Validation accuracy chỉ được sử dụng để theo dõi quá trình huấn luyện. Ngoài accuracy, nhóm báo cáo số tham số có thể học, thời gian huấn luyện và bộ nhớ GPU đỉnh.

Các kết quả được tổng hợp trên 3 seed. Với $n_{seed} = 3$, các chênh lệch nhỏ hơn độ lệch chuẩn giữa các seed không được xem là có ý nghĩa.

So sánh với baseline MLP/CNN. Accuracy được tổng hợp dưới dạng trung bình $\pm$ độ lệch chuẩn mẫu trên 3 seed. Số tham số có thể học cũng được báo cáo để đối chiếu độ phức tạp giữa các kiến trúc.

5.3.4 Môi trường thực nghiệm

Các thí nghiệm E3 được thực hiện cục bộ trên laptop với CPU Intel Core i9-13900H, RAM 16 GB DDR4 và GPU NVIDIA GeForce RTX 3050 Laptop 4 GB. Môi trường phần mềm gồm Python 3.12.3, PyTorch 2.15.0 (CUDA 13.2).

Thời gian huấn luyện và bộ nhớ GPU đỉnh được ghi nhận trong từng lần chạy. Các chỉ số này chỉ có giá trị so sánh *tương đối* giữa các cấu hình trên cùng phần cứng và trong điều kiện thực nghiệm tương đương.

5.4 Kết quả thực nghiệm

5.4.1 So sánh các cấu hình LSTM/GRU

Bảng 5.3 tổng hợp kết quả của bốn cấu hình LSTM/GRU sau 30 epoch huấn luyện. Test accuracy được báo cáo dưới dạng trung bình ± độ lệch chuẩn mẫu trên ba seed. Thời gian huấn luyện là trung bình trung vị thời gian mỗi epoch của từng lần chạy. Bộ nhớ GPU đỉnh được báo cáo bằng giá trị trung bình trên ba seed.

Cấu hình	Accuracy (%)	No. Params	s/epoch	GPU (MB)
LSTM-Row	$63,99 \pm 0,23$	106 826	$8,11 \pm 0,26$	255,69
LSTM-Patch4	$66,84 \pm 0,82$	103 754	$8,03 \pm 0,06$	313,54
GRU-Row	$64,64 \pm 0,42$	81 994	$7,83 \pm 0,18$	245,25
GRU-Patch4	$\mathbf{67,85 \pm 0,76}$	78 922	$7,92 \pm 0,07$	296,12

Bảng 5.3: Kết quả thực nghiệm của bốn cấu hình LSTM/GRU.

So sánh LSTM và GRU. Khi sử dụng cùng phương pháp Row, GRU đạt test accuracy trung bình ở mức64,64%, cao hơn LSTM 0,65 điểm phần trăm. Với Patch4, GRU đạt 67,85%, cao hơn LSTM 1,01 điểm phần trăm. GRU cho kết quả tốt hơn trên cả hai cách biểu diễn, tuy nhiên với kết quả này trên ba seed, chưa đủ cơ sở để khẳng định sự vượt trội ổn định về accuracy.

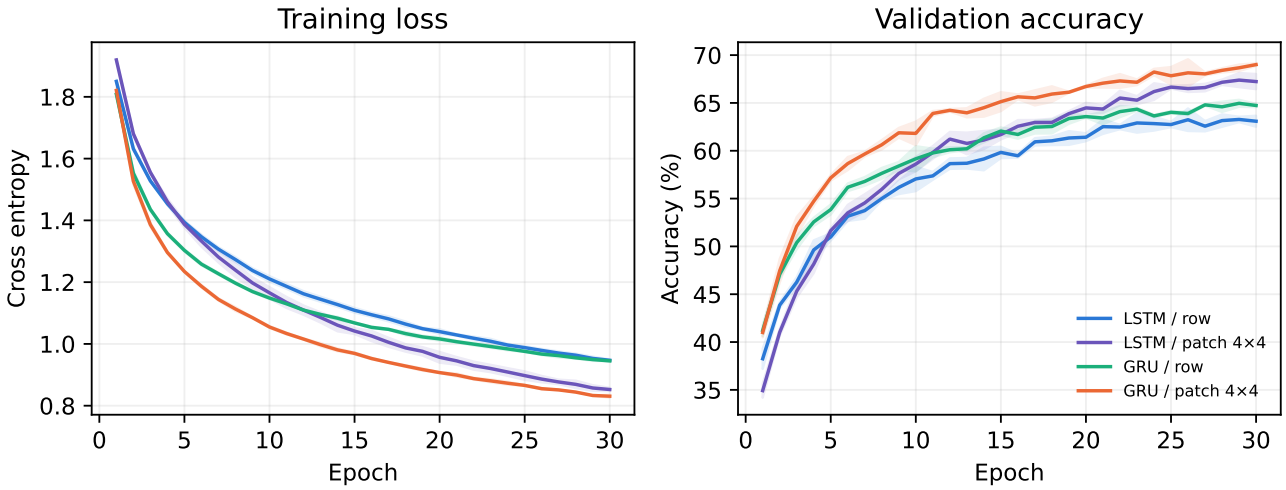
Về số lượng tham số, GRU sử dụng ít hơn LSTM 24 832 tham số trên cả hai tokenizer. Thời gian huấn luyện trung bình của GRU cũng thấp hơn LSTM, với 7,83 so với 8,11 giây/epoch trên Row và 7,92 so với 8,03 giây/epoch trên Patch4. Đồng thời, GRU có mức bộ nhớ GPU đỉnh thấp hơn LSTM trong cả hai trường hợp.

So sánh Row và Patch4. Khi chuyển từ Row ($N = 32$) sang Patch4 ($N = 64$), test accuracy của LSTM tăng từ 63,99% lên 66,84%, tương ứng 2,85 điểm phần trăm. Đối với GRU, accuracy tăng từ 64,64% lên 67,85%, tương ứng 3,21 điểm phần trăm. Mức cải thiện của Patch4 lớn hơn độ biến thiên giữa các seed và xuất hiện trên cả hai kiến trúc.

Mặc dù Patch4 tạo chuỗi dài gấp đôi Row, thời gian huấn luyện mỗi epoch không tăng rõ rệt trong thực nghiệm. Tuy nhiên, mức bộ nhớ GPU đỉnh tăng từ 255,69 lên 313,54 MB đối với LSTM và từ 245,25 lên 296,12 MB đối với GRU. Điều này cho thấy biểu diễn Patch4 yêu cầu nhiều bộ nhớ hơn trong cấu hình huấn luyện được khảo sát.

Nhìn chung, GRU-Patch4 đạt accuracy cao nhất (67,85%) và có ít tham số nhất (78 922) trong cả bốn cấu hình.

Quá trình hội tụ. Bên cạnh các chỉ số tổng hợp trong Bảng 5.3, Hình 5.4 minh họa diễn biến training loss và validation accuracy của bốn cấu hình qua 30 epoch huấn luyện.



Hình 5.4: Diễn biến training loss (trái) và validation accuracy (phải) của bốn cấu hình LSTM/GRU qua 30 epoch huấn luyện trên CIFAR-10. Đường biểu diễn giá trị trung bình trên ba seed;vùng tô mờ xung quanh thể hiện độ lệch chuẩn. Các cấu hình được phân biệt theo recurrent cell (LSTM, GRU) và phương pháp biểu diễn chuỗi (Row, Patch4).

Training loss của tất cả cấu hình đều giảm tương đối ổn định trong 30 epoch. Trong đó, GRU-Patch4 giảm loss nhanh chóng và đạt giá trị thấp nhất ở cuối quá trình huấn luyện.

Validation accuracy của các cấu hình nhìn chung tăng dần theo epoch, mặc dù có một số dao động nhỏ. GRU-Patch4 duy trì kết quả cao nhất trong phần lớn quá trình huấn luyện, tiếp theo là LSTM-Patch4. Hai cấu hình sử dụng Patch4 đạt accuracy cao hơn Row trên cùng kiến trúc, phù hợp với kết quả đánh giá trên tập test.

Nhìn chung, các mô hình đều có xu hướng hội tụ và chưa thể hiện dấu hiệu overfitting nghiêm trọng trong phạm vi 30 epoch. Tuy nhiên, validation accuracy vẫn có xu hướng cải thiện ở các epoch cuối, nên chưa thể khẳng định các mô hình đã hội tụ hoàn toàn.

5.4.2 So sánh với baseline MLP/CNN

Bảng 5.4 đối chiếu kết quả của bốn cấu hình LSTM/GRU với hai baseline MLP và MiniVGG từ E1. Các mô hình được đánh giá trên cùng 10 000 ảnh test CIFAR-10, sử dụng trọng số tại epoch cuối cùng. Test accuracy được báo cáo dưới dạng trung bình ± độ lệch chuẩn mẫu trên ba seed.

Mô hình	Test Accuracy (%)	Params
MLP (E1)	$51,17 \pm 0,23$	199 082
MiniVGG (E1)	$\mathbf{79,62 \pm 0,25}$	37 338
LSTM-Row	$63,99 \pm 0,23$	106 826
LSTM-Patch4	$66,84 \pm 0,82$	103 754
GRU-Row	$64,64 \pm 0,42$	81 994
GRU-Patch4	$67,85 \pm 0,76$	78 922

Bảng 5.4: So sánh test accuracy và số tham số của các mô hình LSTM/GRU với baseline MLP và MiniVGG từ E1. Accuracy được báo cáo dưới dạng trung bình ± độ lệch chuẩn mẫu trên ba seed.

Cả bốn cấu hình LSTM/GRU đều đạt test accuracy cao hơn MLP (51,17%), với mức chênh lệch từ 12,82 đến 16,68 điểm phần trăm. Trong đó, GRU-Patch4 đạt mức cải thiện lớn nhất (16,68 điểm phần trăm), đồng thời số lượng tham số chỉ khoảng 78 922 so với 199 082 của MLP, tương ứng với mức giảm khoảng 60,4% số tham số.

MiniVGG đạt test accuracy cao nhất (79,62%), vượt tất cả các cấu hình hồi quy từ 11,77 đến 15,63 điểm phần trăm. Đáng chú ý, MiniVGG chỉ có 37 338 tham số, là cấu hình sử dụng ít tham số nhất trong cả thí nghiệm. Kết quả này tiếp tục cho thấy tăng số lượng tham số không đồng nghĩa với tăng hiệu quả phân loại.

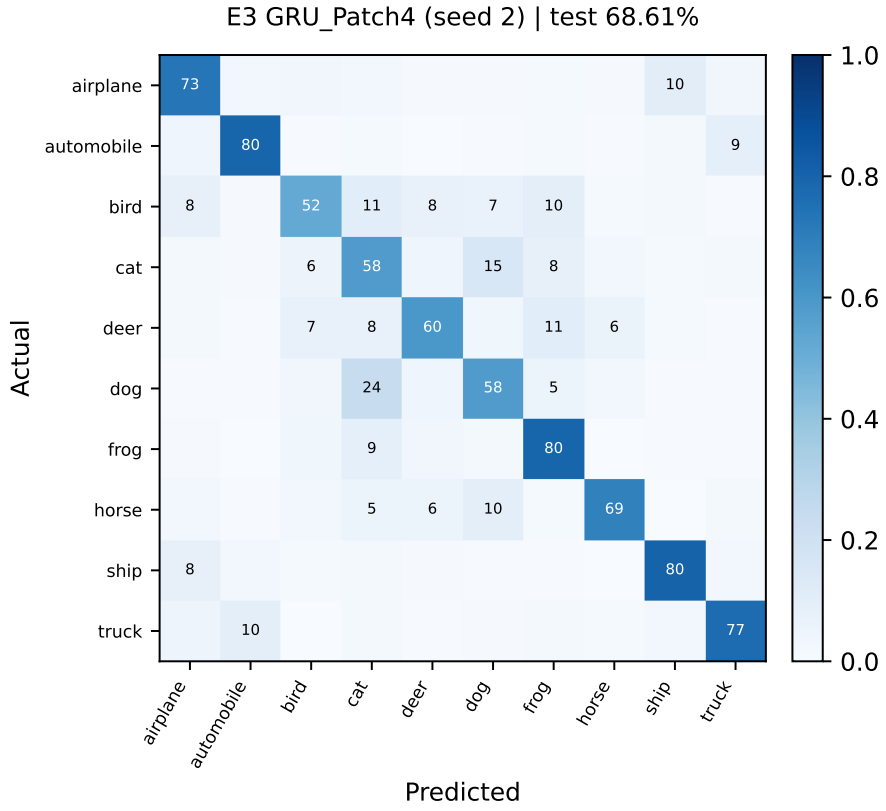
Sự khác biệt về hiểu quả phân loại có thể liên quan đến cách các kiến trúc khai thác thông tin ảnh. MLP xử lý ảnh dưới dạng vector trải phẳng, LSTM và GRU tích lũy thông tin qua chuỗi token, trong khi MiniVGG sử dụng các lớp tích chập để học đặc trưng không gian cục bộ. Trên CIFAR-10, cơ chế tích chập của MiniVGG cho hiệu quả phân loại tốt hơn các kiến trúc hồi quy được khảo sát.

Tuy nhiên, accuracy không trực tiếp phản ánh khả năng nắm bắt phụ thuộc dài hạn. Bên cạnh đó, các baseline E1 và mô hình E3 có một số khác biệt về cấu hình huấn luyện, nên không thể quy toàn bộ chênh lệch kết quả cho riêng kiến trúc mô hình.

5.4.3 Phân tích lỗi

GRU-Patch4 đạt độ chính xác kiểm thử trung bình cao nhất trong bốn cấu hình E3, với $67,85 \pm 0,76\%$ trên ba seed. Nhóm lựa chọn mô hình GRU-Patch4 ở seed 2, đạt test accuracy

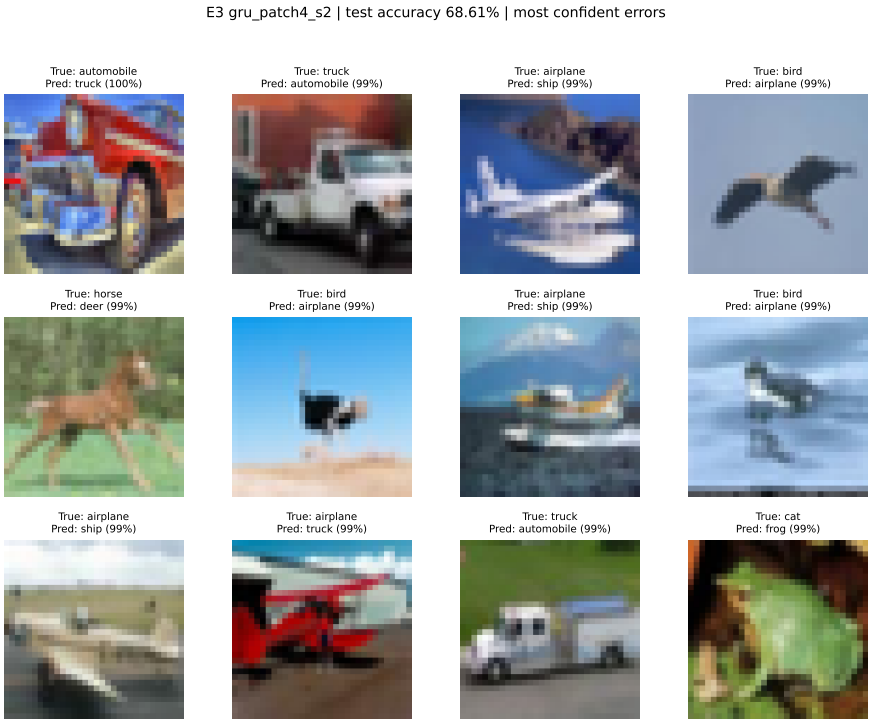
68, 61%, để minh họa ma trận nhầm lẫn và các trường hợp dự đoán sai có độ tin cậy cao. Việc lựa chọn này chỉ phục vụ phân tích lỗi của một lần huấn luyện cụ thể; các kết quả so sánh hiệu năng giữa các cấu hình vẫn được tổng hợp trên cả ba seed.



Hình 5.5: Ma trận nhầm lẫn của GRU-Patch4 (seed 2) trên tập test CIFAR-10, chuẩn hóa theo từng lớp thực tế. Hàng biểu diễn nhãn thực, cột biểu diễn nhãn dự đoán; các giá trị trong ma trận là tỷ lệ phần trăm được làm tròn.

Hình 5.5 cho thấy hiệu quả phân loại có sự khác biệt giữa các lớp. Các lớp *automobile*, *frog* và *ship* đều đạt tỷ lệ nhận diện ở mức cao (80%), trong khi *bird* có tỷ lệ thấp nhất (52%). Hai lớp *cat* và *dog* cùng đạt 58%.

Các cặp nhầm lẫn thường xuyên gồm: *dog* ↔ *cat* (24% và 15%),*deer* → frog (11%), *airplane* thành *ship* (10%), *automobile* thành *truck* (9%) và *truck* thành *automobile* (10%). Các kết quả này cho thấy mô hình gặp khó khăn khi phân biệt một số lớp có đặc trưng hình ảnh tương đồng.



Hình 5.6: Mười hai ảnh dự đoán sai có độ tin cậy cao của GRU-Patch4 (seed 2) trên tập test CIFAR-10. Mỗi ảnh hiển thị nhãn thực (True), nhãn dự đoán (Pred) và xác suất dự đoán của mô hình.

Hình 5.6 minh họa các trường hợp dự đoán sai với xác suất từ 99% đến 100%. Một số lỗi xuất hiện giữa các đối tượng có hình dạng hoặc bối cảnh tương tự, chẳng hạn *automobile&truck*, *airplane&ship* và *horse&deer*. Các trường hợp *bird* bị nhận nhầm thành *airplane* cũng cho thấy mô hình có thể nhầm lẫn giữa những đối tượng xuất hiện trên nền trời.

5.5 Nhận xét và thảo luận

Ảnh hưởng của recurrent cell. GRU đạt test accuracy cao hơn LSTM trên cả hai phương pháp tokenization, với mức chênh lệch 0,65 điểm phần trăm đối với Row và 1,01 điểm phần trăm đối với Patch4. Đồng thời, GRU sử dụng ít hơn 24 832 tham số, có thời gian huấn luyện và bộ nhớ GPU đỉnh thấp hơn LSTM. Tuy nhiên, với chỉ ba seed, chênh lệch accuracy chưa đủ lớn để khẳng định GRU luôn vượt trội so với LSTM.

Ảnh hưởng của tokenization. Patch4 cải thiện test accuracy so với Row 2,85 điểm phần trăm đối với LSTM và 3,21 điểm phần trăm đối với GRU. Mặc dù số token tăng từ 32 lên 64, thời gian huấn luyện mỗi epoch không tăng rõ rệt, trong khi bộ nhớ GPU đỉnh tăng ở cả hai kiến trúc. Kết quả cho thấy cách biểu diễn ảnh thành chuỗi có ảnh hưởng nhất định đến hiệu quả phân loại và chi phí bộ nhớ của mô hình hồi quy.

So sánh với mô hình không hồi quy. Cả bốn cấu hình LSTM/GRU đều đạt accuracy cao hơn MLP nhưng thấp hơn MiniVGG. LSTM và GRU xử lý tuần tự các token, sử dụng hidden state để tích lũy thông tin từ những bước trước. Ngược lại, MLP xử lý ảnh trải phẳng, còn MiniVGG khai thác các đặc trưng không gian thông qua tích chập. Kết quả cho thấy cơ chế hồi quy có thể khai thác thông tin giữa các token để phân loại, nhưng chưa đạt hiệu quả của CNN trong thiết lập thực nghiệm này.

Hạn chế. (i) Mỗi cấu hình chỉ được huấn luyện với ba seed, nên việc đánh giá độ ổn định còn hạn chế. (ii) Các mô hình sử dụng chung siêu tham số và chỉ được huấn luyện trong 30 epoch, chưa tối ưu riêng cho từng kiến trúc. (iii) Thực nghiệm chỉ được thực hiện trên CIFAR-10, chưa đánh giá trên các tập dữ liệu hoặc chuỗi có phụ thuộc thời gian thực. (iv) Chưa có thí nghiệm đối chứng nhằm tách riêng ảnh hưởng của thứ tự token khỏi ảnh hưởng của cách biểu diễn ảnh. (v) Thời gian huấn luyện và bộ nhớ GPU chỉ được đo trên một môi trường phần cứng, nên chỉ có giá trị so sánh mang tính tham khảo.

Chương 6

KẾT LUẬN

6.1 Kết quả chính

E1. Thực nghiệm trên ba kiến trúc SoftmaxClassifier, MLP và MiniVGG cho thấy khả năng khai thác cấu trúc không gian có ảnh hưởng lớn đến hiệu quả phân loại ảnh CIFAR-10. MiniVGG đạt độ chính xác kiểm thử 79,62 ± 0,26%, cao hơn MLP (51,17 ± 0,23%) và Softmax (32,61 ± 0,98%), dù có ít tham số hơn MLP. Kết quả cho thấy các phép tích chập và chia sẻ trọng số giúp học biểu diễn phù hợp hơn với dữ liệu ảnh trong phạm vi thực nghiệm.

E2. Bản Multi-Head Self-Attention tự viết cho kết quả tương đương `nn.MultiheadAttention` (chênh lệch trung bình $\leq$ 0,18 điểm, nhỏ hơn nhiễu giữa các seed), trong khi bản PyTorch nhanh hơn 4–17% mỗi epoch. Cách tạo token ảnh hưởng mạnh hơn nhiều: với cùng encoder gần 0,8–0,9 triệu tham số, CNN-stem đạt 83,0%, Patch 4×4 72,4%, Hàng 69,0% và Patch 8×8 67,1% trên CIFAR-10. Bỏ positional embedding làm giảm khoảng 7 điểm.

E3. Cả bốn cấu hình LSTM/GRU đều đạt accuracy cao hơn MLP nhưng chưa vượt qua MiniVGG. Trong đó, GRU-Patch4 đạt kết quả tốt nhất ($67,85 \pm 0,76\%$), đồng thời có số tham số thấp nhất (78 922) trong nhóm hồi quy.

So với Row, Patch 4×4 cải thiện accuracy 2,85 điểm phần trăm đối với LSTM và 3,21 điểm phần trăm đối với GRU, nhưng yêu cầu bộ nhớ GPU lớn hơn. Kết quả cho thấy cả kiến trúc recurrent cell và phương pháp biểu diễn ảnh thành chuỗi đều ảnh hưởng đến hiệu quả phân loại.

6.2 Hạn chế

Các thực nghiệm vẫn có một số hạn chế.

Thứ nhất, mỗi cấu hình chỉ được huấn luyện với ba seed và số epoch cố định, nên khả năng đánh giá độ ổn định và mức độ hội tụ còn hạn chế. Các siêu tham số cũng chưa được tối ưu riêng cho từng kiến trúc.

Thứ hai, toàn bộ thực nghiệm chỉ sử dụng CIFAR-10, chưa đánh giá khả năng tổng quát hóa sang các tập dữ liệu khác.

Thứ ba, các bài thực hành được thực hiện trên những môi trường phần cứng khác nhau. Do đó, các chỉ số thời gian huấn luyện và bộ nhớ GPU chỉ phù hợp để so sánh các cấu hình trong cùng điều kiện thực nghiệm, không dùng để đối chiếu trực tiếp chi phí tính toán giữa E1, E2 và E3.

Cuối cùng, các phân tích lỗi chủ yếu sử dụng checkpoint của một seed đại diện, nên chưa phản ánh đầy đủ sự biến thiên của hành vi dự đoán giữa các lần huấn luyện.

TÀI LIỆU THAM KHẢO

[1] Samira Abnar and Willem Zuidema. Quantifying attention flow in transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4190–4197, 2020.

[2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[3] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[5] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[6] Adrian Rosebrock. Minivggnet: Going deeper with cnns. `https://pyimagesearch.com/2021/05/22/minivggnet-going-deeper-with-cnns/`, 5 2021. Accessed October 30, 2026.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[8] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Gated recurrent units (gru). `https://d2l.ai/chapter_recurrent-modern/gru.html`. Dive into Deep Learning. Truy cập ngày 09/11/2026.

[9] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Long short-term memory (lstm). `https://d2l.ai/chapter_recurrent-modern/lstm.html`. Dive into Deep Learning. Truy cập ngày 09/11/2026.

Phụ lục A

TÀI NGUYÊN VÀ KHẢ NĂNG TÁI LẬP

- GitHub Repository: `https://github.com/broistg/deep-learning-CO5085`
- Trang tổng quan (GitHub Pages): `https://broistg.github.io/deep-learning-CO5085/`
- Trọng số và logit kiểm thử: GitHub Release `weights-v1` của kho lưu trữ (mã băm SHA-256 trong `common/release_manifest.json`).

Bài tập	GitHub Pages	Notebook Google Colab
E1 – Softmax, MLP và CNN	Trang E1	Mở bằng Colab
E2 – Multi-Head Self-Attention và Tokenization	Trang E2	Mở bằng Colab
E3 – LSTM và GRU	Trang E3	Mở bằng Colab

Bảng A.1: Liên kết GitHub Pages và notebook theo từng bài tập

A.1 Tái lập kết quả E1

Notebook tái lập. Notebook `E1/notebooks/E1.ipynb` sử dụng các module trong thư mục `E1/e1lib` và quy trình dữ liệu dùng chung trong `common`. Notebook hỗ trợ hai chế độ:

- `load`: Nạp các kết quả JSON và tệp dự đoán đã lưu, tổng hợp thống kê, sinh lại bảng và hình mà không cần huấn luyện lại mô hình.
- `rerun`: Huấn luyện lại ba kiến trúc với các seed 0, 1, 2 và lưu kết quả vào thư mục riêng để đối chiếu với kết quả đã công bố.

Chạy bằng dòng lệnh. Các lệnh sau được thực hiện từ thư mục gốc của repository sau khi cấu hình môi trường Python phù hợp:

```
python -m unittest discover -s E1/tests -v
python -m e1lib.run --all
python -m e1lib.summary
python -m e1lib.analysis --seed 0
```

Lệnh huấn luyện có thể được bỏ qua nếu các kết quả đã lưu còn hợp lệ. Các tệp JSON trong `E1/results` chứa cấu hình, lịch sử huấn luyện và chỉ số đánh giá; các tệp dự đoán phục vụ việc xây dựng ma trận nhầm lẫn và trực quan hóa lỗi.

A.2 Tái lập kết quả E2

Notebook tái lập (khuyến nghị)

Notebook `E2/notebooks/E2.ipynb` (mở bằng Google Colab) gọi mã trong kho lưu trữ và không chứa logic riêng. Biến `MODE` ở ô đầu tiên chọn một trong hai chế độ:

- `"load"` (mặc định): nạp 30 tệp JSON kết quả đã lưu, sinh lại các bảng và hình, rồi *kiểm tra* các bảng sinh lại trùng từng ký tự với các bảng trong báo cáo. Chạy được trên Colab miễn phí chỉ với CPU, mất vài phút. Ma trận nhầm lẫn và các ảnh lỗi cần thêm logit kiểm thử, được tải từ GitHub Release `weights-v1` và kiểm tra bằng mã băm SHA-256.
- `"rerun"`: huấn luyện lại toàn bộ 30 lần chạy từ đầu (cần GPU; khoảng 2,5 giờ trên RTX 3050 Laptop), giữ kết quả đã công bố trong `E2/results_published/` để đối chiếu. Do GPU, cuDNN và độ chính xác hỗn hợp khác nhau, kết quả chạy lại không trùng từng bit; chúng được đánh giá bằng trung bình mỗi cấu hình so với độ lệch chuẩn đã công bố.

Dữ liệu CIFAR-10 được tải qua thư viện `datasets` (Hugging Face) và chỉ được chấp nhận khi mã băm SHA-256 của toàn bộ ảnh và nhãn trùng với bản TorchVision dùng để tạo phép chia cố định (bản Hugging Face lưu tập huấn luyện theo thứ tự khác nên được sắp xếp lại; TorchVision là nguồn dự phòng).

Chạy bằng dòng lệnh

Mã nguồn E2 nằm trong thư mục `E2/` của kho lưu trữ; các lệnh dưới đây chạy từ thư mục gốc của kho (`PYTHONPATH=.:E2` hoặc `pip install -e .`). Mỗi bước độc lập với bước sau; kết quả JSON đã chạy xong được giữ trong `E2/results/`, nên có thể bỏ qua bước huấn luyện và chỉ sinh lại bảng, hình.

1. Kiểm thử đơn vị (tương đương MSA, hình dạng token, phép chia dữ liệu): `python -m unittest discover -s E2/tests -v`
2. Do nhanh thời gian, bộ nhớ, MACs: `python -m e2lib.bench`
3. Chạy toàn bộ 30 lần huấn luyện (có thể dừng và chạy tiếp, bỏ qua các lần đã xong; khoảng 2,5 giờ trên RTX 3050 Laptop): `python -m e2lib.plan`
4. Bảng và hình học tập: `python -m e2lib.analyze`
5. Bản đồ chú ý: `python -m e2lib.viz -seed 0`

Môi trường của các kết quả trong báo cáo: Python 3.12, PyTorch 2.14.1 (CUDA 12.6), `timm` 1.0.30, GPU NVIDIA GeForce RTX 3050 Laptop 6 GB, CPU AMD Ryzen 5 6600H, RAM 16 GB; thông tin này cũng được ghi trong từng tệp JSON.

A.3 Tái lập kết quả E3

Notebook tái lập (khuyến nghị). Notebook `E3/notebooks/E3.ipynb` sử dụng các module trong `E3/e3lib` và quy trình xử lý dữ liệu dùng chung trong `common`. Biến `MODE` cho phép lựa chọn hai chế độ:

- `load` (mặc định): Nạp kết quả của 12 lần huấn luyện LSTM/GRU đã lưu trong `E3/results`, sinh lại bảng tổng hợp và đường cong huấn luyện, sau đó đối chiếu với kết quả đã công bố. Trọng số mô hình và logits kiểm thử được tải từ GitHub Release `weights-v1`, kiểm tra mã băm SHA-256 và sử dụng để tái tạo ma trận nhầm lẫn cùng các ảnh dự đoán sai.
- `rerun`: Huấn luyện lại bốn cấu hình LSTM/GRU với ba seed (0, 1, 2), lưu kết quả vào thư mục riêng để đối chiếu với kết quả đã công bố. Chế độ này cần GPU để thực hiện hiệu quả và không ghi đè kết quả E1 hoặc E3 ban đầu.

Chạy bằng dòng lệnh.

Mã nguồn E3 nằm trong thư mục `E3/` của kho lưu trữ. Các lệnh dưới đây được thực hiện từ thư mục gốc của repository, sau khi cài đặt các thư viện cần thiết và cấu hình `PYTHONPATH=.:E3`. Các bước có thể thực hiện riêng biệt; nếu đã có kết quả huấn luyện, có thể bỏ qua bước huấn luyện và chỉ thực hiện phân tích.

1. Kiểm thử đơn vị: Kiểm tra hình dạng đầu ra của các tokenizer và mô hình LSTM/GRU, tính hợp lệ của cấu hình, quy trình huấn luyện và các thành phần liên quan. Đồng thời kiểm tra các tiện ích dùng chung. `python -m unittest discover -s E3/tests -v`
   `python -m unittest discover -s common/tests -v`

2. Kiểm tra trạng thái thực nghiệm: Liệt kê 12 lần chạy tương ứng với bốn cấu hình LSTM/GRU và ba seed (0, 1, 2). Lệnh chỉ kiểm tra các tệp kết quả và checkpoint hiện có, không thực hiện huấn luyện. `python -m e3lib.plan -list`

3. Huấn luyện các mô hình: Thực hiện 12 lần huấn luyện với 30 epoch cho mỗi cấu hình và seed. Chương trình bỏ qua các lần chạy đã hoàn tất và có thể tiếp tục từ checkpoint của những lần chạy bị gián đoạn. `python -m e3lib.plan`

   Có thể chỉ chạy một cấu hình bằng tùy chọn `-only`, chẳng hạn `-only gru_patch4`.

4. Tổng hợp kết quả và vẽ đường cong: Đọc các tệp JSON trong `E3/results`, tính trung bình và độ lệch chuẩn trên ba seed, sinh bảng LaTeX và các đường cong training loss, validation accuracy. Kết quả được ghi vào thư mục riêng để không ghi đè kết quả gốc. `python -m e3lib.analyze -out E3/regenerated`

5. Phân tích lỗi phân loại: Sử dụng logits kiểm thử đã lưu của mô hình GRU-Patch4 (seed 2) để tạo ma trận nhầm lẫn, thống kê độ chính xác theo lớp và trực quan hóa 12 ảnh dự đoán sai có độ tin cậy cao nhất.

   ```
   python -m e3lib.diagnostics
   -run-id gru_patch4_s2
   -out E3/regenerated
   ```

   Bước này yêu cầu tệp `E3/artifacts/gru_patch4_s2/logits_test.pt`. Có thể sử dụng notebook ở chế độ `load` để tự động tải và xác minh artifacts từ GitHub Release `weights-v1`.

Các kết quả trong báo cáo sử dụng Python 3.12.3, PyTorch 2.15.0 (CUDA 13.2) và GPU NVIDIA GeForce RTX 3050 Laptop 4 GB. Thông tin môi trường thực nghiệm được lưu trong các tệp JSON tương ứng.

A.4 Khai báo sử dụng công cụ AI

Nhóm lên ý tưởng và tự viết bản gốc của mã. Công cụ AI (ChatGPT / Claude) hỗ trợ cải thiện và rà soát mã, xây dựng bộ kiểm thử và bản nháp báo cáo. Nhóm đã tự chạy toàn bộ thực nghiệm, kiểm tra các số liệu đưa vào báo cáo với tệp kết quả gốc, và chịu trách nhiệm về toàn bộ nội dung.